Real-Time IoT Sensor Data Pipeline

Streaming ingestion, validation and abnormal-reading detection on Microsoft Azure

Project Planning & Management · Literature Review · Requirements Gathering

Table of Contents

Part 1: Project Planning & Management . . . **3**
1.1 Project Proposal . . . 3
1.2 Project Plan . . . 4
1.3 Task Assignment and Roles . . . 5
1.4 Risk Assessment and Mitigation Plan . . . 6
1.5 Key Performance Indicators (KPIs) . . . 7
1.6 Budget and Cost Control . . . 8
1.7 Progress Tracking and Team Coordination . . . 9
1.8 Additional Technical Risks . . . 10

Part 2: Literature Review . . . **11**
2.1 Industrial Case Studies . . . 11
2.2 Comparison of the Case Studies . . . 13
2.3 Key Findings from the Research . . . 13
2.4 Limitations of the Case-Study Review . . . 14
2.5 Technical Comparison of Azure Services . . . 14
2.6 Differences from Existing Solutions and Research Gaps . . . 17
2.7 Technology Justification . . . 18
2.8 Technologies Named in the Brief: MQTT, Kafka and Spark Streaming . . . 19

Part 3: Requirements Gathering . . . **20**
3.1 Stakeholder Analysis . . . 20
3.2 User Needs . . . 21
3.3 Data Requirements . . . 21
3.4 Use Cases and User Stories . . . 24
3.5 Functional Requirements . . . 25
3.6 Pipeline Workflow . . . 28
3.7 Validation, Transformation and Anomaly-Detection Rules . . . 29
3.8 Dashboard and Alerts Requirements . . . 31
3.9 Prioritization and Traceability . . . 31
3.10 Non-Functional Requirements and Quality Standards . . . 32
3.11 Assumptions and Future Extensions . . . 37
3.12 Supplementary Non-Functional Requirements . . . 38

Part 4: Evaluation and Improvement . . . **39**
4.1 Lecturer Feedback and Evaluation . . . 39
4.2 Suggested Improvements . . . 39
4.3 Final Grading Criteria . . . 40

References . . . **41**

Part 1: Project Planning & Management

This part presents the project proposal, the plan, the assignment of roles, the risk assessment, the key performance indicators (KPIs) and the budget for the Real-Time IoT Sensor Data Pipeline. It is a Data Engineering project on Microsoft Azure that streams and processes simulated industrial sensor data in near real time.

Project category	Data Engineering
Project type	Real-time data streaming and processing
Implementation platform	Microsoft Azure
Team size	6 members

1.1 Project Proposal

1.1.1 Project Overview

The Real-Time IoT Sensor Data Pipeline simulates the monitoring of industrial machines using continuously generated sensor data. Industrial machines produce readings such as temperature, pressure and vibration. Monitoring these readings helps identify abnormal operating conditions and supports timely maintenance decisions.

The project uses Python to generate simulated sensor events and Microsoft Azure services to ingest, process and store the data. A live dashboard presents the main metrics and highlights abnormal readings as they happen; Power BI is used for the historical and analysis pages.

1.1.2 Project Objectives

1. Generate simulated IoT sensor data using Python, including normal, abnormal and deliberately invalid events.
2. Ingest sensor events through Azure Event Hubs.
3. Keep an unmodified copy of every received event in a raw store, independent of validation (Event Hubs Capture).
4. Process and validate incoming data using Azure Stream Analytics.
5. Detect abnormal sensor readings using predefined thresholds.
6. Store processed data in Azure Blob Storage for historical analysis.
7. Provide a live dashboard of sensor readings and abnormal events, using the method confirmed by a proof of concept, and Power BI reports for history.
8. Evaluate the pipeline using performance and data-quality metrics, within a fixed Azure budget.

1.1.3 Project Scope

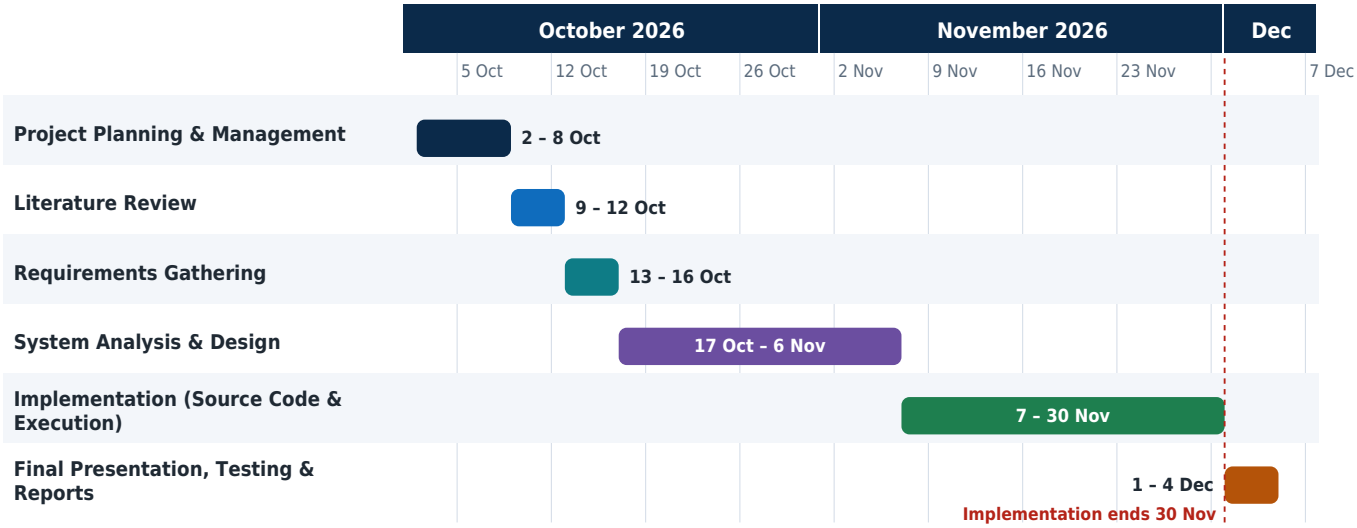
In scope	Out of scope
Simulating industrial sensors and generating continuous events	Connecting to physical industrial sensors
Streaming ingestion and near-real-time processing	Machine-learning models for predictive maintenance
Raw-data capture, data validation and basic error handling	A production-grade industrial monitoring platform
Threshold-based abnormal-event detection	Automatic control of industrial machines
Historical data storage with a documented retrieval method	Enterprise-level security beyond the basic controls in Section 3.10.6, and geo-redundant disaster recovery
A live dashboard, historical reports and performance testing at prototype scale	Load tests above the prototype scale (see Future Extensions, Section 3.11.2)
Documentation and the final project demonstration	

The project focuses on a manageable prototype that demonstrates the complete data pipeline within the available time, resources and Azure budget.

1.2 Project Plan

1.2.1 Timeline and Gantt Chart

Project phases run from 2 October to 4 December 2026. Implementation ends on 30 November.



Phase	Start	End	Days	Primary owners
Project Planning & Management	2 Oct 2026	8 Oct 2026	7	Shahd Saber
Literature Review	9 Oct 2026	12 Oct 2026	4	Jana Sherif, Youssef Said
Requirements Gathering	13 Oct 2026	16 Oct 2026	4	Jana Islam, Salma Islam, Ayman Fathy
System Analysis & Design	17 Oct 2026	6 Nov 2026	21	All members
Implementation (Source Code & Execution)	7 Nov 2026	30 Nov 2026	24	All members
Final Presentation & Testing & Reports	1 Dec 2026	4 Dec 2026	4	All members

1.2.2 Milestones and Deliverables

Milestone	Target date	Expected deliverable
M1: Planning completed	8 Oct 2026	Project proposal, scope and initial KPIs
M2: Literature Review completed	12 Oct 2026	Literature research, comparative analysis and sources
M3: Requirements completed	16 Oct 2026	Stakeholders, user stories, functional and non-functional requirements
M4: System design completed	6 Nov 2026	Architecture and design; live-dashboard proof of concept recorded and method chosen
M5: Implementation completed	30 Nov 2026	Simulator, ingestion, raw capture, processing, storage and live dashboard
M6: Final presentation and reports	4 Dec 2026	Integrated testing results, final report and demonstration

1.2.3 Resource Allocation

Resource	Purpose
Python	Generate simulated sensor events; Last Seen health check; evaluation scripts
Azure Event Hubs	Ingest and transport events; Capture writes the raw copy to Blob Storage
Azure Stream Analytics	Validate, deduplicate, process and analyze incoming data
Azure Blob Storage (ADLS Gen2)	Store raw, valid, alert, aggregate and rejected records
Live dashboard (method per proof of concept)	Show current readings, status and alerts as they arrive
Power BI	Historical and analysis reports built on the stored files
GitHub	Manage source code and team collaboration
Visual Studio Code	Develop and test the project
Microsoft documentation	Support implementation and troubleshooting

1.3 Task Assignment and Roles

Each member has a primary responsibility. All members cooperate during integration, testing and the final presentation.

1.3.1 Shahd Saber — Team Leader and Project Planning

- Define the project overview, objectives and scope.
- Prepare the timeline, milestones and Gantt chart.
- Assign tasks and monitor progress.
- Identify project risks and coordinate mitigation plans.
- Define and track the project KPIs.
- Own the Azure budget: track spending and make sure resources are stopped after each session (Section 1.6).
- Ensure that all deliverables are completed before the deadline.

Main deliverable: Project proposal, project plan, risk register, KPI summary and budget plan.

1.3.2 Jana Sherif — Literature Research

- Research three to four similar projects or existing solutions.
- Examine their architectures, workflows and technologies.
- Identify their main advantages and limitations.
- Collect reliable references and prepare the research report.

Main deliverable: Literature research report.

1.3.3 Youssef Said — Comparative and Technical Analysis

- Compare the researched solutions and relevant technologies.
- Evaluate performance, scalability, complexity and cost.
- Identify limitations in existing approaches.
- Justify the selected technology stack.
- Run the live-dashboard proof of concept and document the result.

Main deliverable: Technology comparison and selection report.

1.3.4 Jana Islam — Stakeholder Analysis and Data Requirements

- Identify the main stakeholders and their needs.
- Define the sensor data fields, formats and units.
- Specify data ingestion requirements and expected event frequency.
- Document the project's main use cases and user stories.

Main deliverable: Stakeholder analysis and data requirements document.

1.3.5 Salma Islam — Functional Requirements and Workflow

- Define the functional requirements.
- Document the pipeline workflow.
- Specify data validation, transformation and anomaly-detection rules.
- Define dashboard requirements and prioritize project features.

Main deliverable: Functional requirements and pipeline workflow document.

1.3.6 Ayman Fathy — Non-Functional Requirements and Quality

- Define performance, reliability, scalability, security and usability requirements.
- Establish data-quality standards.
- Prepare test cases and measurable acceptance criteria.
- Evaluate the system's performance and document test results.

Main deliverable: Non-functional requirements and testing report.

1.3.7 Shared Responsibilities

All team members contribute to reviewing the final architecture and technology selection, integrating and testing the complete pipeline, preparing the final presentation and demonstration, and reviewing the documentation and ensuring successful submission.

1.4 Risk Assessment and Mitigation Plan

The table lists the main risks that could affect project completion or system performance. Reliability, security and data-quality requirements that mitigate several of these risks are defined in Section 3.10.

ID	Risk	Likelihood	Impact	Mitigation plan
R1	Python simulator fails or stops generating events	Medium	High	Add error handling, logging and restart instructions.
R2	Azure services are unavailable or difficult to configure	Medium	High	Test connectivity early and follow the official documentation.
R3	Events are lost or duplicated	Medium	High	Use a unique event_id per event, deduplicate within a defined time window, and reconcile generated, raw, processed and stored counts.
R4	Sensor data contains missing or invalid values	High	Medium	Apply validation rules and separate invalid records for review.
R5	Reading-level processing latency exceeds the target	Medium	High	Measure latency, reduce unnecessary processing and monitor event throughput.
R6	Abnormal readings are incorrectly classified	Medium	High	Define clear thresholds and test normal, abnormal and boundary values.
R7	Storage or dashboard integration fails	Medium	High	Test each component independently before full integration.
R8	Azure cost exceeds the project budget	Medium	High	Fixed budget cap, cost alerts, short test sessions, and stopping resources after each session (Section 1.6).

ID	Risk	Likelihood	Impact	Mitigation plan
R9	The live-dashboard method is not available. The Stream Analytics Power BI output is deprecated and closed to new customers [6][7].	High	High	Proof of concept by M4 (6 Nov) that compares live methods; choose the cheapest method that passes; a second live method is kept as fallback (Section 2.7.1).
R10	Duplicates that arrive after the deduplication window or after a job restart are not caught	Medium	Medium	Document the window; run a post-run duplicate check on stored data; no exactly-once guarantee is claimed.
R11	Event Hubs Capture is not available on the chosen tier	Low	High	Capture needs the Standard tier or higher [8]; confirm the tier before deployment. The simulator audit log is the documented fallback (Section 3.3.4).

1.5 Key Performance Indicators (KPIs)

KPIs evaluate whether the project meets its performance, reliability, data-quality and management objectives. The targets below are the same values used in the non-functional requirements (Section 3.10). Every KPI is measured in a controlled test, not over a long production period.

KPI	What it measures	Target	Measurement
Reading and alert latency	Time from the event timestamp until the single processed record (or its alert) is written to the output store	Valid readings: P95 of 5 s or less (NFR-P1). Alerts: P99 of 10 s or less (NFR-P2)	Compare event timestamp with output write time
Window result delay	Time from the end of a 1-minute or 5-minute window until its aggregate is written	90 s or less after window end (NFR-P6); measured separately from the 5-second target	Compare window end time with output write time
Throughput	Events processed per second	50 events/s baseline for 30 minutes if the budget allows (NFR-P3); optional 100–200 events/s for 10 minutes	Count processed events over the test period
Data loss rate	Generated events not found at the designated outputs after reconciliation	0% unexplained loss	Compare unique event_ids across generated, raw and processed records
Duplicate rate	Events stored more than once	0% in the controlled test (NFR-R2)	Count repeated event_ids in the stored output
Availability (test sessions)	Share of scheduled demo and test time in which the pipeline ran successfully	99.5% or more of session time (NFR-R3)	Successful operating time divided by scheduled session time
Valid-record acceptance	Share of known valid test records that are accepted	At least 99% (NFR-DQ1)	Compare accepted records with known valid inputs
Invalid-record rejection	Share of deliberately invalid test records that are rejected	100% of predefined cases (NFR-DQ2)	Compare rejections with expected outcomes
Threshold detection	Predefined boundary and threshold test cases detected correctly	100% of predefined cases (AD-01 to AD-05)	Compare alerts with expected outcomes
Injected-anomaly detection	Recall and false positives on labelled injected anomalies	Recall 95% or more; false positives 5% or less (NFR-DQ7)	Evaluation script against ground-truth labels
Live dashboard refresh	Time from output update to visible dashboard update	10 s or less with the live method chosen by the proof of concept (NFR-P5)	Observe and record update times
Usability	First-time users find the critical alert on the overview page	At least 4 of 5 testers within 30 s (NFR-U1)	Short observed test with five testers

The data-quality rows measure different things: valid-record acceptance checks that good records are not wrongly rejected, and invalid-record rejection checks that bad records are not wrongly accepted. In the same way, the threshold-case test (fixed expected outcomes) is separate from recall and false-positive rate (a statistical test on injected anomalies).

1.5.1 KPI Measurement

1. Generate a known number of sensor events, each with a unique event_id.
2. Include normal, abnormal, invalid and duplicate records.
3. Record the number of events generated, stored raw, processed and stored.
4. Measure reading latency, window delay and dashboard delay.
5. Compare actual results with the expected outcomes.
6. Record the results and document any failures.

The final report includes the actual KPI results, the test conditions and any limitations, so that project performance is demonstrated with measured evidence rather than assumptions.

1.6 Budget and Cost Control

Azure Stream Analytics is billed while a job is running, and Event Hubs is billed per throughput unit and hour. Running 50 simulated devices continuously would use paid resources without adding value for the demonstration, so the project runs in short, planned sessions.

1.6.1 Budget cap and planned usage

The amounts below are planning assumptions. They must be checked against the Azure pricing calculator and the credit actually available to the team before the first deployment.

Item	Plan
Maximum budget	A fixed cap set by the team leader and approved by the supervisor before deployment. Proposed starting value: USD 50 for the whole project [to be confirmed against available credit]
Development runs	Short runs with 5–10 simulated devices, about 15–30 minutes each
Formal test sessions	Planned sessions only: the 50 events/s baseline test (50 devices, 30 minutes) if the budget allows; the duplicate and restart test; the latency and window test; and, as an optional extra, a 100–200 events/s test (10 minutes)
If the budget does not allow the baseline	Run the same tests at a lower rate, record the rate actually used, and state it in the results
Expected running hours	About 40 hours of Stream Analytics job time in total [estimate; update with real usage]
Cost alerts	Azure cost budget with email alerts at 50%, 80% and 100% of the cap
Tier choice	Smallest Event Hubs and Stream Analytics settings that meet the tests. Event Hubs Capture is not available on the Basic tier, so Standard is the minimum [8]
Live-dashboard cost	Included in the cap. The proof of concept compares the cost of each live method before one is chosen

1.6.2 Resource shutdown plan

1. After every session: stop the simulator, stop the Stream Analytics job, and confirm that the job state is Stopped.
2. Outside test weeks: keep only Blob Storage (low cost) and delete or scale down the Event Hubs namespace if the tier allows.
3. After the final demonstration (planned by 4 December): export the results and test evidence, then delete the whole Azure resource group.
4. The team leader checks Azure Cost Management at the end of each week and records the spending in the progress notes.
5. If 80% of the cap is reached, only the remaining required acceptance tests are run.

Part 1 (continued)

1.7 Progress Tracking and Team Coordination

Section 1.3 assigns the work. This section sets the routine the team leader uses to follow progress, spot delays early and keep every member informed. It covers the project phases of 2 October – 4 December 2026 and adds no cost.

Mechanism	How it works	Who	When
Task board	One card per deliverable from Section 1.3 on a shared board (for example GitHub Projects). Each card has an owner, a due date linked to milestones M1–M6 and a status: To do, In progress, In review or Done.	Shahd Saber, card owners	Updated whenever work changes
Weekly team meeting	A 20-minute meeting: what was finished, what comes next and what blocks the work. Decisions and actions are written in the progress notes.	All members	Once a week
Short check-in	A short message in the team chat: finished, planned, blocked. Used mainly during implementation (7–30 November).	All members	Twice a week
Milestone review	The deliverable is compared with Section 1.2.2 on the milestone date and marked on time or late.	Shahd Saber	8 Oct, 12 Oct, 16 Oct, 6 Nov, 30 Nov, 4 Dec
Review before merge	Code and documents are changed through GitHub pull requests and checked by at least one member other than the author.	All members	Every change
Budget check	Azure spending is read from Cost Management and recorded in the progress notes (Section 1.6.2).	Shahd Saber	End of each week

Progress measures and escalation

- The team leader reports progress as the share of cards in Done for the current phase, the number of open blockers and the number of late tasks.
- A task that is more than 2 days late, or blocked, is raised at the next meeting, or at once if it threatens a milestone. The team leader then reassigns it, arranges help from another member or reduces its scope.
- Scope is reduced in MoSCoW order (Section 3.5): Nice items first, then Should items. Must items are protected.
- If a milestone is missed, the supervisor is told the reason and the new date.

1.8 Additional Technical Risks

Section 1.4 lists the general risks. This section adds the technical risks that concern sensors, the network, the processing job and late data, so that each has its own mitigation and test. R3 (events lost or duplicated) and R5 (latency above target) in Section 1.4 remain valid and are linked in the second table.

ID	Risk	Likelihood	Impact	Mitigation plan	Test
R12	A sensor or device stops sending (connection lost or device failure)	Medium	High	The Last Seen check flags a device as possibly offline after 30 s of silence (FR-21, Section 3.7.4). Devices are pre-registered, so a device that never sends is also found. The dashboard shows offline devices.	NFR-R5
R13	Network interruption between the simulator and Azure	Medium	High	The simulator retries failed sends with a limited back-off and logs every failure (FR-06). Event Hubs buffers events for the retention period, so processing can resume (Standard tier: up to 7 days [8]). Counts are reconciled after each run (Section 3.4).	NFR-R1, NFR-R6
R14	Events arrive late or out of order	High	Medium	Event time, arrival time and processing time are kept apart (Section 3.3.5). Events up to 60 s late are counted in their window; later events are stored, flagged is_late and left out of aggregates; alerts still apply.	NFR-R4, AC-6
R15	The Stream Analytics job stops or fails during a run	Medium	High	Azure Monitor charts show backlog, errors and SU utilization (NFR-O1). Events stay in Event Hubs, so the job resumes from its last checkpoint after a restart. The restart test is part of the acceptance run.	NFR-R1, NFR-R6, AC-1

Risk named in the brief	Handled in the register by	Requirement or test
Loss of the connection to a sensor	R12, R13	NFR-R5, NFR-R1
Loss of data	R3, R13, R15	NFR-R1, data loss rate KPI (Section 1.5)
Late arrival of data	R14, R5	NFR-R4, NFR-P1, AC-6

Part 2: Literature Review

This part reviews four industrial case studies and one Azure reference solution, compares the candidate Azure services layer by layer, and records why the chosen stack fits this project. Sources are numbered [1]–[8] and listed with links in the References section at the end of the document. The lecturer's feedback and the final grading criteria are in Part 4.

2.1 Industrial Case Studies

2.1.1 Grupo Bimbo — Real-Time Manufacturing Data Monitoring

Industry	Food manufacturing
Technology provider	Microsoft Azure
Source	Microsoft Customer Story — Grupo Bimbo [1]

Problem and solution. Grupo Bimbo, a multinational baking company, needed better visibility into the performance of its factory production lines. Data from industrial equipment had to be consolidated and made accessible to factory employees and management. The company implemented an Azure-based architecture that collects production data from Programmable Logic Controllers (PLCs), integrates it with other systems and makes it available for analytics and visualization.

Architecture. Production Equipment → PLCs → Azure IoT Services → Azure Data Factory → Azure SQL Database and Azure Data Lake → Power BI. Azure Data Explorer is also used to correlate quality information with production scheduling data. This is a simplified representation, not a claim that every component forms one strictly sequential chain.

Technologies used. PLCs (collect equipment data), Azure IoT Services (integrate production data), Azure Data Factory (orchestrate integration), Azure SQL Database (consolidate structured data), Azure Data Lake (analytical storage), Azure Data Explorer (correlate production and quality data), Power BI (performance reports shown on factory floors and other devices).

Advantages. A consolidated view of production performance; operational data available through dashboards; analysis of production and quality information; industrial data collection integrated with analytical storage and reporting.

Limitations and challenges. Integrating different factory systems requires careful data modeling; many services must be managed; dashboard usefulness depends on data quality. These are challenges inferred from the documented design, not necessarily problems Grupo Bimbo reported.

Relevance to our project. The case shows how equipment data can be collected, consolidated, stored and visualized. Our project applies similar principles with Python-generated readings, a streaming platform, historical storage and a dashboard. The published case does not establish that Grupo Bimbo uses threshold-based anomaly detection like ours.

2.1.2 XTO Energy — IoT-Based Monitoring of Remote Industrial Assets

Industry	Oil and gas
Technology provider	Microsoft Azure
Source	Microsoft Customer Story — XTO Energy [2]

Problem and solution. XTO Energy, a subsidiary of ExxonMobil, operates geographically distributed assets. Before its IoT solution, technicians often recorded well readings manually and transferred them to a central office; limited connectivity made timely access difficult. XTO collects measurements from existing equipment sensors and sends them to Azure for preprocessing, storage, visualization and analysis.

Architecture. Equipment Sensors → Azure IoT Edge → Azure IoT Hub → Azure Stream Analytics → Azure Cosmos DB and Azure Data Lake. Azure Time Series Insights is used for storing, visualizing and querying time-series data, with

further integrations for employee applications and dashboards.

Workflow. Sensors measure temperature, pressure and flow; IoT Edge gateways collect and route the data; IoT Hub provides cloud connectivity; Stream Analytics preprocesses and transforms it; Cosmos DB and Data Lake store it; Time Series Insights supports visualization; dashboards help technicians identify equipment that needs attention. Azure Data Factory supports data movement.

Advantages. Near-real-time access to remote measurements; less manual data collection; historical analysis; helps technicians prioritize facilities; an architecture designed for unreliable connectivity.

Limitations and challenges. Network interruptions at remote sites; integration work for older equipment; large sensor volumes need careful storage and processing design; many cloud services add configuration and operations work.

Relevance to our project. This is one of the closest matches to our project: it collects temperature, pressure and flow, processes incoming data, stores history and gives near-real-time visibility. Our project uses simulated events and adds explicit validation rules, vibration measurements, threshold-based detection and time-window aggregations.

2.1.3 3M — Edge Analytics for Manufacturing Anomaly Prediction

Industry	Manufacturing
Technology provider	Microsoft Azure
Source	Microsoft Customer Story — 3M manufacturing plant [3]

Problem and solution. At one plant, 3M wanted to detect potential problems before they caused significant downtime. It had to combine two production systems whose data streams could arrive at different times, and limited connectivity made sending all data to the cloud difficult. 3M built an edge solution with Azure IoT Edge and Azure SQL Edge to synchronize and analyze data locally, correlating equipment measurements with product information and applying analytics and machine learning to predict problems.

Architecture. Production Equipment and Product Data → Azure IoT Edge → Azure SQL Edge → Local Analytics and Machine Learning → Cloud Analytics. The implementation also integrates with cloud machine-learning tools and can transfer data to cloud services.

Advantages. Less dependence on continuous cloud connectivity; processing close to the equipment; combines data from several production systems; can reduce latency; shows how streaming operational data supports predictive maintenance.

Limitations and challenges. Streams that arrive at different times need synchronization; machine-learning predictions need suitable training data and evaluation; edge deployments must be maintained; the architecture is more complex than simple threshold-based monitoring.

Relevance to our project. The case is relevant to abnormal-event detection. The distinction is that our project detects abnormal readings with predefined thresholds, whereas 3M uses analytics and machine learning to predict problems. Machine learning could be added later but is not needed for the first implementation.

2.1.4 EMSTEEL — Real-Time Manufacturing Intelligence

Industry	Steel manufacturing
Technology provider	Microsoft
Source	Microsoft Customer Story — EMSTEEL [4]

Problem and solution. EMSTEEL needed better visibility into manufacturing operations and a way to connect operational data with analytics. It wanted to monitor machine status, production rates and energy consumption while supporting advanced analytics and predictive maintenance. It uses Microsoft Fabric Real-Time Intelligence to process live data from equipment and shop-floor sensors, Azure Databricks for advanced analytics, and an Eventhouse in Fabric to connect older systems with modern applications.

Architecture. Manufacturing Equipment and Sensors → Microsoft Fabric Real-Time Intelligence → Eventhouse → Analytics and Operational Monitoring. Azure Databricks adds an analytics layer for patterns, predictive maintenance and production optimization. The representation does not imply that every flow passes through the Eventhouse in this exact order.

Advantages. Visibility into live manufacturing conditions; analysis of production and energy metrics; operational data connected with advanced analytics; a path toward predictive maintenance; helps integrate older systems with newer applications.

Limitations and challenges. Legacy integration needs care; advanced analytics needs data preparation and expertise; cloud processing and storage need configuration and resource management; the architecture may be excessive for a small project with simulated data.

Relevance to our project. EMSTEEL is a recent example of real-time intelligence in industry and is relevant to our monitoring dashboard and future historical-data extension. It also matters for the live-dashboard decision (Section 2.7.1), because Fabric Real-Time Intelligence is the service Microsoft recommends in place of the deprecated Power BI streaming output [6]. We adopt the principle of processing sensor data and exposing operational metrics, without reproducing the Microsoft Fabric architecture.

2.2 Comparison of the Case Studies

The four cases share a common goal: improving operational decisions by making industrial data more accessible and useful. Each emphasizes a different aspect of the pipeline.

Criterion	Grupo Bimbo	XTO Energy	3M	EMSTEEL
Industry	Food manufacturing	Oil and gas	Manufacturing	Steel manufacturing
Primary objective	Monitor production performance	Monitor remote equipment	Predict manufacturing problems	Monitor operations and optimize production
Data sources	Production-line PLCs	Well-equipment sensors	Equipment data and product information	Equipment and shop-floor sensors
Real-time / near-real-time	Data integration and operational reporting	Yes	Local data processing and analytics	Yes
Main processing approach	Azure data integration and analytics	Azure Stream Analytics	Edge analytics and machine learning	Fabric Real-Time Intelligence and Databricks
Historical data	Yes	Yes	Local storage and cloud integration	Eventhouse and analytical data
Anomaly detection / prediction	Not the main focus of the cited case	Operational condition monitoring	Predictive anomaly detection	Predictive maintenance and pattern analysis
Visualization	Power BI	Time-series tools and dashboards	Analytical tools	Real-time intelligence and analytics
Closest project requirement	Operational metrics and dashboard	Sensor ingestion and historical storage	Abnormal-event detection	Real-time monitoring and future analytics

> **Interpretation**
> XTO Energy is the most relevant to sensor ingestion and storage, 3M is the strongest example of predictive anomaly detection, Grupo Bimbo demonstrates production-data integration and dashboards, and EMSTEEL illustrates a modern real-time analytics architecture.

2.3 Key Findings from the Research

2.3.1 Event ingestion is a separate architectural layer. The cases show that collecting data and processing it are distinct responsibilities (XTO uses IoT Edge and IoT Hub to collect and route, and Stream Analytics to preprocess). **Implication:** the Python simulator sends records to a dedicated ingestion service, Azure Event Hubs, instead of implementing processing and storage itself.

2.3.2 Historical storage is essential. Industrial data is more useful when current readings can be compared with earlier behavior. XTO retains large amounts of history, and Grupo Bimbo uses consolidated data and analytical storage. **Implication:** the pipeline keeps historical records so users can analyze trends, compare machines and investigate earlier abnormal events.

2.3.3 Abnormal-event detection can use different approaches. Threshold-based detection flags readings above configured limits; statistical analysis finds unusual deviations; machine learning finds complex patterns and predicts failures (3M). **Implication:** the project starts with explicit thresholds for temperature, pressure and vibration; more advanced detection can follow if enough data exists.

2.3.4 Edge processing can reduce dependence on connectivity. 3M processes data locally and XTO designed for connectivity interruptions. **Implication:** edge computing is a possible future extension, not needed for simulated events.

2.3.5 Real-time dashboards turn data into operational information. Grupo Bimbo exposes performance data through Power BI and EMSTEEL monitors operations with real-time intelligence. **Implication:** the live dashboard shows current temperature, pressure and vibration per machine, machine status, the number of abnormal readings, average temperature over a time window, and historical trends per machine.

2.3.6 The services behind a live dashboard change over time. The NRI reference solution (2019) feeds Power BI directly from Stream Analytics [5], but Microsoft has since deprecated that output for new customers [6][7], and EMSTEEL uses Fabric Real-Time Intelligence [4]. **Implication:** the live-dashboard method is verified by a proof of concept before the architecture is frozen.

2.4 Limitations of the Case-Study Review

- **Vendor-published sources.** The sources are mainly official Microsoft customer stories. They identify documented architectures and technologies but are vendor accounts, not independent evaluations.
- **Different operational requirements.** An oil and gas company monitoring remote wells does not have the same needs as a food manufacturer monitoring production lines.
- **Different anomaly-detection methods.** Some cases focus on operational monitoring, others on predictive analytics.
- **Scale difference.** The reported architectures are much larger than this student project. The purpose is to identify transferable principles, not to reproduce their infrastructure.
- **Age and link status of sources.** Customer stories and the NRI article were published in different years, so service names and limits may have changed. All links were opened on 20 October 2026 (see References).

2.5 Technical Comparison of Azure Services

2.5.1 Purpose and method

This section compares the Azure services that can implement each layer (ingestion, processing, storage and dashboard) and records why the chosen stack fits this project. It answers how the candidates compare on performance, scalability and complexity; how this project differs from existing solutions; and why Event Hubs, Stream Analytics, Blob Storage (ADLS Gen2) and a live dashboard with Power BI reports are chosen.

- One further Azure reference solution (Section 2.5.2) is used alongside the four case studies. It is a consultancy description of intended benefits, not independent evidence of performance.
- *Not described* means the feature was not found in the material reviewed, not that the solution lacks it.
- The ratings in Section 2.5.4 are qualitative team judgments based on the sources and general Azure documentation. The weights reflect the project brief and can be adjusted.
- Product limits, tiers and prices change over time and must be re-checked in current Microsoft documentation before implementation (Section 3.10.12).

2.5.2 Azure reference solution

Solution	Machine Monitoring IoT Solution with Azure Services and Power BI (NRI Digital, June 2019) [5]
Type	Consultancy reference solution on Azure
How it works	Azure Functions collect events; Event Hubs capture and persist them (Capture to Azure Data Lake); Stream Analytics performs anomaly detection and aggregation; results stream to a Power BI dataset through the Power BI REST API.
Strengths	Closest to this project's stack; fully managed services; claims sub-minute reporting latency; history retained in the data lake.
Limitations	The latency claim is not backed by a published test; the article dates from 2019 and relies on the Power BI streaming output that is now deprecated; the dataset keeps only 200,000 rows (first in, first out); validation and rejected-record handling are not described.

2.5.3 Technology comparison by layer

Ingestion

Option	Strengths	Weaknesses	Best fit
Azure Event Hubs	Fully managed partitioned event stream built for high-volume telemetry; Capture can persist events to Blob Storage or a data lake (Standard tier or higher [8])	Throughput units and quotas must be sized; partition count is fixed on the Standard tier	Streaming telemetry on Azure **(chosen)**
Azure IoT Hub	Per-device identity and device management; optimized for IoT	Extra features unused by a simulator; more setup	Real devices that need individual identities
Azure Service Bus	Reliable enterprise messaging with queues, topics and sessions	Designed for message-by-message business workflows, not high-volume telemetry	Commands and transactional workflows

Stream processing

Option	Performance	Scalability	Complexity	Notes
Azure Stream Analytics	Continuous SQL-like queries; latency depends on the query and Streaming Units (SUs)	Scale by adding SUs; parallelism follows input partitions	Low: managed service, language similar to T-SQL	Native Blob Storage output; arrival time is the default timestamp unless event time is set. The Power BI output is deprecated [6]
Azure Databricks (Spark Structured Streaming)	Micro-batch model; latency set by the trigger interval	Scales across cluster nodes	Medium: workspace and cluster, code in Python	Flexible for machine learning and complex logic
Azure Functions (Event Hubs trigger)	Event-driven code; low latency per batch of events	Scales automatically with the number of partitions	Medium: windows, deduplication and state must be coded by hand	Fully custom logic; dashboard output must be coded

Storage and visualization

Layer	Option	Strengths	Weaknesses	Role here
History	Blob Storage / ADLS Gen2	Low-cost storage for large raw history; open formats such as Parquet; Stream Analytics can write to it	A file store, not a database: retrieval by device and time depends on the folder layout (Section 3.3.4)	Raw, valid, alert, aggregate and rejected data **(chosen)**
Serving	Azure SQL	Familiar SQL; good for aggregates, alerts and dashboards	Cost grows with raw volume; not needed for the prototype	Optional extension
Serving	Azure Data Explorer	Fast analytics on time-series telemetry	Additional service and query language	Considered, not selected
Serving	Azure Cosmos DB	Low-latency document store (used in the XTO case)	Cost grows with large raw volumes	Considered, not selected

Layer	Option	Strengths	Weaknesses	Role here
Live view	Power BI streaming dataset (Stream Analytics output)	Native output; non-developers can build reports	Deprecated; closed to new customers since 31 Oct 2024 and stops working on 31 Oct 2027; about one push per second; 15 KB packets; 200,000-row limit [6][7]	Only if the team's subscription can still use it (proof of concept)
Live view	Fabric Real-Time Intelligence	Microsoft's recommended replacement for the deprecated output [6]; used in the EMSTEEL case [4]	Additional service and capacity cost; more rework of the processing logic	Proof-of-concept candidate
Live view	Python live dashboard (for example Streamlit or Dash)	Lowest cost; full control; reads the latest valid, alert and health files	Not Power BI; the team builds and maintains it	Proof-of-concept candidate and fallback
Live view	Azure Managed Grafana	Rich time-series panels	An additional paid service outside the project brief	Fallback option
History reports	Power BI over Blob Storage files	Required by the brief; scheduled refresh; easy for analysts	Refresh is in minutes, not seconds	Historical and analysis pages **(chosen)**

2.5.4 Weighted decision matrices

Each option is scored from 1 (poor) to 5 (excellent). The weights favor fit with the project brief and implementation effort, because the schedule and the Azure scope are fixed. The scores are qualitative team judgments, so the matrices support the decision rather than prove it.

Ingestion criterion (weight)	Event Hubs	IoT Hub	Service Bus
Fit with the project brief (25%)	5	3	2
Implementation effort (25%)	4	3	4
Throughput and scalability (20%)	5	3	2
Low operational burden (15%)	5	5	5
Built-in security features (15%)	4	5	4
Weighted score (out of 5)	**4.60**	**3.60**	**3.25**

Processing criterion (weight)	Stream Analytics	Azure Databricks (Spark)	Azure Functions
Fit with the project brief (25%)	5	4	3
Implementation effort (25%)	5	3	3
Latency (15%)	4	3	4
Scalability (15%)	4	5	4
Low operational burden (10%)	5	3	4
Flexibility for complex logic (10%)	3	5	5
Weighted score (out of 5)	**4.50**	**3.75**	**3.60**

Azure Databricks is the strongest alternative where custom Python logic or machine-learning models are needed. Neither alternative is required by the current scope.

2.6 Differences from Existing Solutions and Research Gaps

2.6.1 Differences between this project and existing solutions

Aspect	Existing solutions reviewed	This project
Data quality	Focused on moving and storing data; validation and rejected-record handling are not described in the reviewed material	Validation rules for completeness, validity, timeliness and uniqueness; well-formed rejected records are kept with a reason, and every received event is kept in a raw store
Quality targets	Qualitative statements, such as sub-minute latency in the Azure reference solution	Measurable targets: P95 reading latency, window delay, throughput, availability during test sessions and data-loss rate (Section 3.10)
Detection testing	Detection accuracy is not reported in the reviewed material	The simulator injects labelled anomalies; recall and false-positive rate are measured against ground truth
Live and history	Separate services for live monitoring and historical analysis (for example Time Series Insights and Data Lake in the XTO case)	One Stream Analytics job writes the stored outputs that feed both the live dashboard and the historical reports; an optional view compares current behavior with history (UC-04)
Scale and effort	Enterprise-scale architectures with many services (edge, Databricks, Fabric)	A compact managed stack sized for a prototype and a fixed budget, with a documented test per requirement
Security	Treated as a platform feature; not detailed in the case summaries	A basic baseline (TLS, send-only credentials, least privilege, no secrets in code), each with a test; stronger controls are optional

2.6.2 Research gaps and how the project addresses them

#	Gap observed in the material reviewed	How this project addresses it
G1	Data-quality rules and rejected-record handling are rarely documented together with the streaming design	Explicit rule set in the query, rejected store with reasons, raw store for everything received, tests for each rule
G2	Performance claims are qualitative and seldom tied to a test method	Each non-functional requirement has a metric, threshold and test; reading latency and window delay are measured separately
G3	Detection quality is rarely reported with ground-truth testing	Labelled anomaly injection and an evaluation script giving recall and false-positive rate
G4	Live and historical analysis are often handled by separate services	A single managed pipeline whose stored outputs feed live and historical views, with an optional current-versus-history comparison
G5	Public case studies give little guidance on reliability behavior such as restart and replay	Acceptance scenarios for restart with no loss, deduplication of replayed events inside a defined window, and late-event handling
G6	Reference solutions often rely on a single dashboard connector, without checking whether it is still supported	A proof of concept compares live-dashboard methods before the architecture is frozen, and records the result

These are gaps in the material reviewed, which is a small sample. They are observations, not claims about the whole field.

2.7 Technology Justification

Layer	Choice	Why	Alternative and trade-off	Revisit when
Ingestion	Azure Event Hubs	Managed and partitioned; built for high-volume telemetry; Capture keeps an independent raw copy; highest matrix score	IoT Hub adds per-device identity that a simulator does not need	Real devices need individual identities
Processing	Azure Stream Analytics	SQL-like language; native output to Blob Storage; lowest effort within the schedule	Databricks offers more flexibility but needs cluster setup; Functions needs hand-coded windowing and deduplication	Logic outgrows SQL, or models must run in the stream
Storage	Azure Blob Storage (ADLS Gen2)	Low-cost and open format; supports later analysis; readable by Python and Power BI	Azure Data Explorer is faster for time-series queries but adds a service; Azure SQL is a possible serving store	Interactive history queries become heavy
Live dashboard	Method chosen by the proof of concept (Section 2.7.1)	A live view is a Must requirement; the Power BI output of Stream Analytics is deprecated for new customers [6]	Power BI streaming output, Fabric Real-Time Intelligence, a Python live dashboard, or Azure Managed Grafana	The proof of concept fails or a method's cost or licence changes
History reports	Power BI	Required by the brief; reads files from Blob Storage	Python reports if the licence is not available	Custom visuals beyond Power BI limits are needed
Simulator	Python	Fast to write, rich libraries, reproducible with seeds and labelled output	Dedicated load tools generate higher rates	Load beyond one machine is needed

2.7.1 Live-dashboard finding and proof of concept (before the architecture is frozen)

> **Finding from the source check**
> Microsoft's documentation states that real-time streaming in Power BI is deprecated. New customers could not create Stream Analytics jobs with a Power BI output after 31 October 2024, and existing jobs that use it stop on 31 October 2027 [6][7]. The original design assumed this connector would be available. Because the live dashboard is now a Must requirement, the team does not rely on it without testing.

The live dashboard stays a core deliverable; only the technology behind it is open. The team tests the candidate methods below with a few sample records and records the result.

Method	Path	Main check in the proof of concept
A. Power BI streaming output	Stream Analytics → Power BI	Can the team's subscription still create the output? Refresh time, row limit and licence. It stops working on 31 Oct 2027 [6]
B. Fabric Real-Time Intelligence	Event Hubs → Eventstream → KQL database → real-time dashboard or Power BI	Free or trial capacity, cost against the budget, effort to repeat the validation rules
C. Python live dashboard	Latest valid, alert and health files in Blob Storage → Streamlit or Dash	Refresh time with files written every few seconds; effort to build
D. Azure Managed Grafana	Blob Storage or a serving store → Grafana panels	Cost per hour; setup time; data-source support

1. **Test (by M4, 6 November).** Send a few sample records through each candidate the team's account supports. Record whether it connects, the visible refresh time, the licence or capacity needed and the cost.
2. **Decide.** Choose the cheapest method that works, refreshes in 10 s or less (NFR-P5) and fits the budget. Keep a second passing method as a fallback where time allows. Document the choice.
3. **If none reaches 10 s.** Document the measured time and agree it with the supervisor. A live method is still delivered; scheduled refresh from Blob Storage (minutes) is not a replacement.
4. **History reports and record.** Power BI reads the stored files for the historical pages in every case. The proof-of-concept result is added to the final documentation as evidence.

Part 2 (continued)

2.8 Technologies Named in the Brief: MQTT, Kafka and Spark Streaming

The brief asks which technologies the similar solutions use, for example MQTT, Kafka and Spark Streaming, as far as the sources really use them. The technology lists recorded in Sections 2.1 and 2.5.2 describe Azure services. None of them includes MQTT or Kafka, and only EMSTEEL [4] names a Spark-based platform (Azure Databricks). The first table records what each source names; the second compares the three technologies from the brief with the chosen stack.

Source	Technologies named
Grupo Bimbo [1]	PLCs, Azure IoT services, Azure Data Factory, Azure SQL Database, Azure Data Lake, Azure Data Explorer, Power BI
XTO Energy [2]	Azure IoT Edge, Azure IoT Hub, Azure Stream Analytics, Azure Cosmos DB, Azure Data Lake, Azure Time Series Insights, Azure Data Factory
3M [3]	Azure IoT Edge, Azure SQL Edge, local analytics and machine learning, cloud machine-learning tools
EMSTEEL [4]	Microsoft Fabric Real-Time Intelligence, Eventhouse, Azure Databricks
NRI reference solution [5]	Azure Functions, Azure Event Hubs with Capture, Azure Stream Analytics, Power BI REST API

Azure Databricks is a managed Apache Spark service. The EMSTEEL story as summarized here does not say that Spark Streaming itself is used, so this document does not claim it.

Technologies from the brief compared with the chosen stack

Technology	What it is	Azure counterpart	Performance	Scalability	Complexity	Decision
MQTT	Lightweight publish/subscribe protocol for small devices	IoT Hub accepts MQTT from devices; Event Hubs uses AMQP, the Kafka protocol and HTTPS	Low overhead per message	Needs a broker or IoT Hub	Adds protocol and device-identity setup	Not used: the simulator sends JSON with the Event Hubs SDK (Section 2.5.3)
Apache Kafka	Distributed, partitioned event log with consumer groups	Event Hubs offers a Kafka-compatible endpoint (Standard tier or higher)	High throughput	Grows with partitions and brokers	High if self-hosted: a cluster must be run	Not used: Event Hubs gives the same partition model as a managed service
Spark Streaming	Micro-batch stream processing in Python or Scala	Azure Databricks (Section 2.5.3)	Latency set by the trigger interval	Scales across cluster nodes	Medium: workspace, cluster and code	Alternative, not required: revisit if models must run in the stream
Stream Analytics (chosen)	Continuous SQL-like queries	Native Azure service	Latency set by the query and Streaming Units	Scales by adding SUs	Low: managed, language like T-SQL	Chosen (Section 2.7)

The counterparts and ratings come from general Azure documentation and are qualitative team judgments, like those in Section 2.5.4. Protocol support and tier limits must be re-checked before implementation (Section 3.10.12).

Part 3: Requirements Gathering

3.1 Stakeholder Analysis

Stakeholders are the people, teams and systems that use, operate, fund or are affected by the pipeline. They are listed by their influence over the design and their interest in the system.

Stakeholder	Role in the project	Main interest	Influence / Interest
Machine Operators	Work on the factory floor and run the machines day to day	Immediate alerts when a machine behaves abnormally so they can react safely	Medium / High
Maintenance Engineers	Inspect, repair and service machines	Early warning of faults and trends that indicate wear (for example rising vibration)	High / High
Plant / Production Managers	Oversee production lines, output and downtime	Live machine status, availability and summary KPIs for decisions	High / High
Quality Assurance Team	Ensure product quality and process compliance	Evidence that machines operated within allowed limits during production	Medium / Medium
Data Analysts / BI Team	Analyze historical data and build dashboards	Clean, well-structured historical data and Power BI access	Medium / High
Data Engineers (project team)	Build and maintain the pipeline	Reliable ingestion, scalable processing and clear data contracts	High / High
IT / Cloud Administrators	Manage infrastructure, security and costs	Secure access, controlled cost and monitoring of platform health	Medium / Medium
Executive Management	Sponsor the project and approve investment	Reduced downtime and measurable operational benefit	High / Low

3.1.1 Stakeholder prioritization

- **Key players (high influence, high interest):** Maintenance Engineers, Plant Managers and the Data Engineering team. Their requirements drive the core design and are validated first.
- **Keep informed:** Machine Operators and the BI team. They use the system daily and need clear, reliable outputs.
- **Keep satisfied:** Executive Management and IT Administrators. They need periodic summaries and assurance on cost and security.
- **Monitor:** The Quality Assurance team, whose needs are largely met by the historical store.

3.2 User Needs

User group	Needs	How the system addresses them
Machine Operators	Instant notification of threshold violations; a simple view of current machine status; alerts that are clear and not overly frequent	Threshold-violation detection and a live alert panel; status tiles per machine
Maintenance Engineers	Early detection of abnormal temperature, pressure or vibration; trends over time per device; history of past incidents for root-cause analysis	Windowed aggregations (average, min, max); historical storage and device-level trend charts
Plant / Production Managers	Overview of all machines at a glance; count of active alerts and machines in fault; comparison of current behavior against historical norms	Live dashboard; optional historical-versus-live comparison
Quality Assurance	Proof that machines stayed within operating limits; ability to retrieve data for a given time period or batch	Time-stamped historical store organized by date and device
Data Analysts / BI	Consistent schema and clean data; access to raw and aggregated data; documented data-quality rules	Validated records, curated outputs, README and data dictionary
Data Engineers	Reliable ingestion with no silent data loss; clear handling of invalid, duplicate or late events	Event Hubs buffering, raw store, validation stage and a rejected store
IT / Cloud Admins	Secure connections and least-privilege access; visibility of pipeline health and costs	Managed Azure services, access control, basic monitoring and cost alerts
Executive Management	Evidence of reduced downtime and faster response; concise KPIs rather than raw data	Summary KPIs on the dashboard and a demonstration of abnormal-event detection
All dashboard users	Easy-to-read screens; alerts that are understood quickly; a short time to learn the system; a clear way to run the project	Usability requirements NFR-U1 to NFR-U4 (Section 3.10.8): plain layout, severity shown by label and color, README

3.3 Data Requirements

3.3.1 Data collected from IoT sensors

Each machine (device) emits events containing identification, timing, sensor measurements and status. All devices share one fixed schema. Validation rules for each field are defined in Section 3.7.1.

Field	Type	Unit	Example	Description
event_id	String (UUID)	-	7c9e6679-7425-...-e07fc1f90ae7	Unique identifier of this event, created by the simulator. Never reused. Used for deduplication and reconciliation. Required.
device_id	String	-	MCH-014	Unique identifier of the machine/sensor. Required.
timestamp	DateTime (UTC, ISO 8601)	-	2026-10-16T09:30:05Z	Event time: when the reading was taken at the source. Required.
temperature	Float	°C	78.4	Machine temperature. Required.
pressure	Float	bar	6.2	Operating pressure. Required.
vibration	Float	mm/s	3.1	Vibration intensity. Required.
humidity (optional)	Float	%	45.0	Ambient relative humidity near the machine. Optional.
machine_status	String (enum)	-	RUNNING	Status reported by the device: RUNNING, IDLE, WARNING, FAULT or OFFLINE. Required.

Three separate concepts. The documentation keeps these apart and never overwrites one with another:

Concept	Where it comes from	Meaning
machine_status	Sent by the device (simulator)	What the machine itself reports. Validated against the allowed values and stored as sent.
alert_severity	Calculated by the pipeline (None, Warning, Critical)	The result of comparing the readings with the thresholds.

Concept	Where it comes from	Meaning
possibly_offline	Calculated by the Last Seen check (Section 3.7.4)	True when no reading has arrived from the device for more than 30 seconds.

If a device reports RUNNING while a reading is above the Critical threshold, the record keeps machine_status = RUNNING, alert_severity = Critical and an alert is raised. The dashboard shows both: the reported status on the tile and the alert severity as a separate badge.

Sample event payload (JSON, UTF-8):

```
{
  "event_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "device_id": "MCH-014",
  "timestamp": "2026-10-16T09:30:05Z",
  "temperature": 78.4,
  "pressure": 6.2,
  "vibration": 3.1,
  "humidity": 45.0,
  "machine_status": "RUNNING"
}
```

3.3.2 Example threshold rules

Threshold values are placeholders for the simulation. They are to be confirmed with maintenance engineers and, in a real deployment, set per machine type from manufacturer specifications.

Measurement	Warning level	Critical level
Temperature	Above 85 °C	Above 100 °C
Pressure	Above 8 bar	Above 10 bar
Vibration	Above 6 mm/s	Above 10 mm/s

3.3.3 Data ingestion requirements

Requirement	Specification
Data source	Python IoT-event simulator generating events for multiple virtual machines.
Ingestion channel	Azure Event Hubs, one event per message.
Message format	JSON, UTF-8 encoded, with a fixed schema shared by all devices.
Arrival rate	One event per device every 1 second (configurable from 1 to 10 seconds).
Number of devices	5–10 simulated machines for development runs. 50 machines (50 events/s) for the baseline test and demonstration if the budget allows. Short sessions only.
Expected throughput	About 50 events per second at baseline during a test session if the budget allows. A test at 100–200 events/s for 10 minutes is optional. Higher rates are listed only under Future Extensions (Section 3.11.2).
Estimated volume	Roughly 300 bytes per event. A 30-minute baseline session is about 90,000 events (about 27 MB).
Latency target	Single reading: processed and written to the output store within 5 seconds (P95). Window results are measured separately (Section 3.10.3).
Partitioning key	device_id, so that all events from one machine stay in order within a partition.
Ordering and late data	Events are ordered per device. Late events within 1 minute of arrival are included in window aggregates; later events are stored but flagged and excluded (Section 3.3.5).
Delivery guarantee	At-least-once delivery. Duplicates are detected using event_id within a limited time window (Section 3.7.1, V-10). Exactly-once delivery is not claimed.
Invalid records	Well-formed records that fail validation are never discarded silently; they go to a separate rejected store with the failure reason. Malformed JSON is kept in the raw store (Section 3.3.4).
Data retention	Raw events and processed results are kept in Blob Storage. Event Hubs retention follows the chosen tier (Standard allows up to 7 days [8]), which is enough for the restart test.
Security	Authenticated producers only, encrypted connections and least-privilege access (NFR-SEC1 to SEC6).

3.3.4 Raw data path and historical retrieval

Primary raw path: Event Hubs Capture. The raw store holds messages received by Event Hubs independently of validation. Event Hubs Capture writes them to Blob Storage before and apart from the Stream Analytics job. Capture is not available on the Basic tier, so the Standard tier or higher is required [8]. Whether Capture is enabled is confirmed before the first deployment.

Fallback: simulator audit log. If Capture is not available, the simulator records each attempted send, its event_id, payload, send time and the SDK acknowledgement or error in a local append-only log. After each test, the log is uploaded to Blob Storage and acknowledged sends are reconciled with the downstream outputs.

> **Limits of the fallback.** The audit log is written by the producer. It is not proof that Event Hubs received a message, and it cannot guarantee an exact copy of every event that Event Hubs received. It is a documented alternative that supports reconciliation, not a full guarantee of raw data. If the fallback is used, this limitation is stated in the test results.

Malformed JSON. A message that is not valid JSON cannot be read by the Stream Analytics query. It is preserved in the raw store and counted by the Azure Monitor input-deserialization-error metric. It does not appear in the rejected store, because that store holds well-formed records together with the failed rule.

Historical retrieval. Blob Storage stores files and is not a query engine. Retrieval by device and time period (FR-26) is done through the folder layout:

```
{stream}/device_id={device_id}/{yyyy}/{MM}/{dd}/{HH}/part-*.json
```

where *stream* is one of valid, alerts, aggregates or rejected. To retrieve a period, a person or script reads the folders for the chosen device and dates, using Python (pandas or pyarrow) or the Power BI folder connector. The layout is set in the Stream Analytics output path pattern (to be confirmed in Section 3.10.12). No SQL database is required for the prototype.

3.3.5 Time definitions and late events

Three different times are used and must not be mixed:

Time	Meaning	Source	Used for
Event time	When the reading was taken at the device	timestamp field	Window membership (the query uses TIMESTAMP BY timestamp); future-timestamp check
Arrival time	When Event Hubs received the message	Event Hubs enqueued time	Lateness check; Last Seen check
Processing time	When the pipeline wrote the record	Pipeline clock, saved as processing_time	Latency measurement

Lateness. lateness = arrival time minus event time. An event is late when lateness is more than 60 seconds. A future timestamp is an event time later than arrival time by more than a small clock-skew allowance (5 seconds).

Handling. Events with lateness up to 60 seconds go into the window of their event time. Events later than 60 seconds are still stored in the valid store with is_late = true and can still raise an alert (an abnormal reading is still abnormal), but they are excluded from the final window aggregates. Storing a late event does not mean it appears in an aggregate.

3.4 Use Cases and User Stories

ID	Actor	Scenario	Outcome
UC-01	Machine Operator	Temperature on a machine rises above the critical limit. The pipeline flags the event and it appears on the live alert panel.	The operator sees the alert within seconds and can slow or stop the machine before damage occurs.
UC-02	Maintenance Engineer	The engineer opens the device view and checks 5-minute average vibration for a machine over the past week.	A rising trend is spotted and preventive maintenance is scheduled before failure.
UC-03	Plant Manager	The manager views the dashboard showing machine status counts and active alerts across the plant.	Quick decisions on shift planning and resource allocation.
UC-04	Plant Manager / Analyst	Current behavior of a machine is compared against its historical average (optional extension).	Deviations from normal performance are identified, not only fixed threshold breaches.
UC-05	Quality Assurance	After a quality complaint, QA retrieves all readings for the machines used in a specific time period.	Evidence is available to confirm or rule out machine conditions as the cause.
UC-06	Data Analyst	The analyst queries historical processed data and builds Power BI reports on alert frequency per machine.	Insights on the least reliable machines and common causes of alerts.
UC-07	Data Engineer	The engineer reviews rejected records and their failure reasons.	Faulty sensors or data issues are identified and corrected quickly.
UC-08	Executive	Management reviews summary KPIs such as number of alerts, average response time and uptime.	Evidence of the value delivered by the pipeline.

Reconciliation. After outputs finish, separately count attempted and acknowledged sends, raw messages, valid, rejected, duplicate, malformed and failed messages for the same run. Compare unique event_ids; raw message counts are not unique-event counts and may differ if Capture is unavailable.

3.4.1 Detailed example: UC-01 abnormal reading detection

Primary actor	Machine Operator
Trigger	A sensor event with a measurement above the configured threshold arrives.
Preconditions	The simulator and streaming pipeline are running; thresholds are configured.
Main flow	1. The simulator sends an event to Azure Event Hubs. 2. Event Hubs Capture saves the raw copy. 3. The Stream Analytics job reads and validates the event. 4. The job compares the measurements to the thresholds. 5. A violation is flagged and written to storage. 6. The live dashboard updates and shows the alert for that machine.
Alternative flow	If the record is invalid, it is sent to the rejected store and no alert is raised.
Result	The abnormal event is visible on the live dashboard and kept in history.

3.4.2 User stories

ID	User story	Acceptance idea	Use case
US-01	As a machine operator, I want to see an alert when a reading passes a limit, so that I can react before damage occurs.	Alert appears on the panel with machine, value, limit and severity	UC-01
US-02	As a machine operator, I want the machine's reported status and the alert level shown separately, so that I notice when a machine says RUNNING but a reading is critical.	Both the status tile and the alert badge are visible	UC-01, UC-03
US-03	As a maintenance engineer, I want 5-minute averages per machine over time, so that I can spot wear before failure.	Trend chart per device from aggregate files	UC-02
US-04	As a plant manager, I want the status of all machines and the number of active alerts on one page, so that I can plan the shift.	Overview page shows status counts, active alerts and machines in fault	UC-03
US-05	As a QA officer, I want to retrieve all readings of a machine for a time period, so that I can check a quality complaint.	Files found by device and date folder	UC-05
US-06	As a data analyst, I want to see how often each machine raised alerts, so that I can find the least reliable ones.	Alert frequency report	UC-06
US-07	As a data engineer, I want rejected records stored with a reason, so that I can find faulty sensors.	Every rejected record has rule and reason	UC-07
US-08	As a manager, I want a few summary KPIs, so that I can judge the value of the system without reading raw data.	KPI tiles on the dashboard	UC-08
US-09	As a first-time dashboard user, I want to find the machine with a critical alert quickly, so that I do not need training.	Found within 30 s by at least 4 of 5 testers	UC-01, UC-03

3.5 Functional Requirements

This section defines what the system must do. It does not define how fast, how secure or how scalable it must be; those are non-functional requirements (Section 3.10). Priorities follow the MoSCoW method: **Must** (required for a working prototype), **Should** (important, included if time allows) and **Nice** (optional extension).

3.5.1 Event generation

ID	Requirement	Priority
FR-01	The system shall include a Python simulator that generates sensor events for multiple virtual machines.	**Must**
FR-02	Each event shall contain a unique event_id, device_id, timestamp (UTC, ISO 8601), temperature, pressure, vibration and machine_status.	**Must**
FR-03	Each event may contain humidity as an optional field.	**Nice**
FR-04	The simulator shall generate normal readings, abnormal readings (above Warning or Critical thresholds), deliberately invalid records, duplicate events and a few malformed messages, so that all pipeline paths can be demonstrated and tested.	**Must**
FR-05	The simulator shall allow the number of devices and the event interval to be configured.	**Should**
FR-06	The simulator shall log its activity and report errors instead of failing silently.	**Should**

3.5.2 Ingestion

ID	Requirement	Priority
FR-07	The simulator shall send each event as one JSON message to Azure Event Hubs.	**Must**
FR-08	All devices shall use the same fixed event schema.	**Must**
FR-09	Events shall be partitioned by device_id so that events from one machine stay in order.	**Should**

3.5.3 Validation and cleaning

ID	Requirement	Priority
FR-10	The system shall validate each event for schema, data type, range, timestamp and allowed status values (Section 3.7.1).	**Must**
FR-11	Each processed record shall be marked valid or invalid, and invalid records shall carry the reason for rejection.	**Must**
FR-12	Well-formed invalid records shall be routed to a separate rejected store and shall never be discarded silently.	**Must**
FR-13	Invalid records shall not trigger alerts and shall not be included in aggregates.	**Must**
FR-14	The system shall drop a repeated event that has the same event_id as an event already seen within the deduplication window (10 minutes).	**Must**
FR-15	Late events (more than 60 seconds after event time) shall be stored and flagged, and excluded from window aggregates. Events within the tolerance shall be included.	**Should**

3.5.4 Real-time processing and anomaly detection

ID	Requirement	Priority
FR-16	The system shall compare temperature, pressure and vibration against configured Warning and Critical thresholds.	**Must**
FR-17	When a threshold is exceeded, the system shall create an alert record containing event_id, device_id, timestamp, the measurement, its value, the limit exceeded and the severity (Warning or Critical).	**Must**
FR-18	When several measurements in one event exceed limits, the event severity shall be the highest of them, and every violated measurement shall be recorded.	**Must**
FR-19	Thresholds shall be defined in configuration, not hard-coded into the logic.	**Should**
FR-20	The system shall calculate average, minimum, maximum and count of readings per device over 1-minute and 5-minute time windows.	**Must**
FR-21	The system shall flag a device as possibly offline when no reading has arrived for more than 30 seconds, using a periodic Last Seen check (Section 3.7.4).	**Should**
FR-22	The system shall produce a machine health summary: reported status, alert severity, last seen time and number of threshold violations per device.	**Must**

3.5.5 Storage

ID	Requirement	Priority
FR-23	The system shall store every received event, including invalid and malformed ones, in an independent raw store (see FR-40).	**Must**
FR-24	The system shall store valid processed records, alert records and window aggregates in Azure Blob Storage.	**Must**
FR-25	The system shall store well-formed rejected records, with their failure reasons, in a separate location.	**Must**
FR-26	Stored records shall be time-stamped and retrievable by device and time period through the documented folder layout (Section 3.3.4), to support traceability (UC-05).	**Must**
FR-27	Historical data shall be retained so that earlier readings and alerts can be analyzed later.	**Must**

3.5.6 Dashboard and alerts

ID	Requirement	Priority
FR-28	The dashboard shall show the current reported status of every machine (Running, Idle, Warning, Fault, Offline) and, separately, its alert severity.	**Must**
FR-29	The dashboard shall show the latest temperature, pressure and vibration per machine.	**Must**
FR-30	The dashboard shall include a live alert panel listing recent alerts with machine, measurement, value and severity.	**Must**
FR-31	The dashboard shall show summary KPIs: number of active alerts, number of machines in Fault and count of abnormal readings.	**Must**
FR-32	The dashboard shall show the average temperature over a defined time window.	**Should**
FR-33	The dashboard shall show historical trend charts per device (for example 5-minute average vibration over time).	**Should**
FR-34	The dashboard shall show alert frequency per machine, for analysts.	**Nice**
FR-35	The dashboard shall show the number and reasons of rejected records, so data engineers can find faulty sensors.	**Nice**
FR-36	The dashboard shall let users compare a machine's current behavior with its historical average (optional extension, UC-04).	**Nice**
FR-37	Alerts shall be visually distinguished by severity (Warning and Critical).	**Should**
FR-42	The dashboard shall be live: new readings, statuses and alerts shall become visible within the refresh target (NFR-P5), using the method confirmed by the proof of concept (FR-41).	**Must**

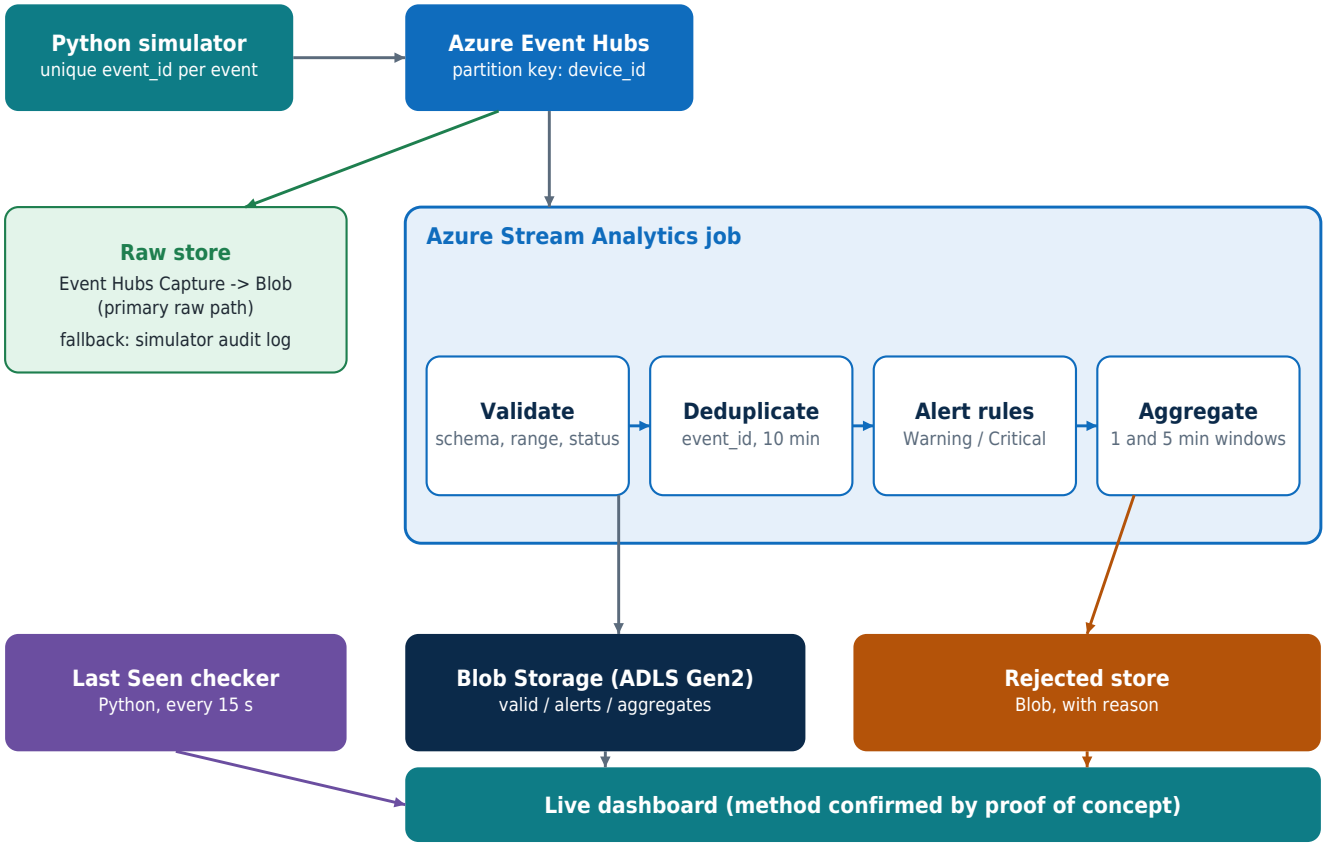
3.5.7 Demonstration, documentation and verification

ID	Requirement	Priority
FR-38	The team shall be able to demonstrate abnormal-event detection end to end: a simulated abnormal reading appears as an alert on the live dashboard and is kept in history.	**Must**
FR-39	The project shall include an architecture diagram and a README explaining the streaming architecture and how to run it.	**Must**
FR-40	The raw store shall be written by a path that is independent of the validation stage. Event Hubs Capture is the primary path. The simulator audit log is the documented fallback and its limits (Section 3.3.4) shall be stated in the test results.	**Must**
FR-41	The team shall complete a live-dashboard proof of concept by M4 (6 November 2026) and record the result, the method chosen and any fallback (Section 2.7.1).	**Must**

3.6 Pipeline Workflow

The pipeline has seven stages. Ingestion is kept separate from processing, as recommended by the literature review (Section 2.3.1). Inside the Stream Analytics stage the data passes through validation, deduplication, threshold detection and window aggregation.

Figure 3.1 — End-to-end pipeline architecture on Microsoft Azure



Green = raw path, independent of validation. Orange = rejected records. Purple = offline detection.

The raw path is independent of the processing path. The live dashboard method is chosen by the proof of concept (Section 2.7.1).

3.6.1 Stage-by-stage description

#	Stage	What happens	Output
1	Event generation	The Python simulator creates events with unique event_ids for each virtual machine, including normal, abnormal, invalid, duplicate and malformed cases.	JSON events
2	Ingestion and raw capture	Events are sent to Azure Event Hubs, one event per message. Capture saves an unmodified copy to the raw store. Event Hubs buffers and delivers events to processing.	Event stream, raw store
3	Validation	Each readable event is checked for schema, type, range, timestamp and status.	Valid or invalid flag
4	Cleaning	Invalid records go to the rejected store with a reason. Duplicates (same event_id inside the window) are dropped. Late events are flagged.	Clean valid stream, rejected records
5	Threshold detection	Valid events are compared with Warning and Critical limits. Violations create alert records.	Alert records
6	Window aggregation	Average, minimum, maximum and count are calculated per device over 1-minute and 5-minute windows, using event time. A health summary per device is produced.	Aggregates, health summary
7	Storage and presentation	Valid, alert, aggregate and rejected data are written to Blob Storage. The live dashboard shows current readings and alerts; Power BI reads the historical files.	Live dashboard, history

3.6.2 Data paths

- **Normal path:** a valid event with no threshold violation is stored as a valid record and counted in aggregates.

- **Alert path:** a valid event with a violation is stored as a valid record, also creates an alert record and appears on the alert panel.
- **Rejected path:** a well-formed invalid event goes to the rejected store with a reason. It raises no alert and is not counted in aggregates.
- **Malformed path:** a message that is not valid JSON stays in the raw store only and is counted by the deserialization-error metric.
- **Duplicate path:** a repeated event_id inside the deduplication window is dropped after the first copy has been processed.
- **Late path:** an event more than 60 seconds late is stored and flagged, can still raise an alert, and is excluded from final aggregates.
- **Offline path:** if a device sends nothing for more than 30 seconds, the Last Seen check flags it as possibly offline.

3.7 Validation, Transformation and Anomaly-Detection Rules

3.7.1 Data validation and cleaning rules

Validation applies to events that can be read as JSON. Malformed messages are handled by the raw path (Section 3.3.4).

Rule	Field	Condition	If it fails
V-01	Schema	All required fields (event_id, device_id, timestamp, temperature, pressure, vibration, machine_status) are present and of the correct type	Reject: missing or invalid field
V-02	device_id	Non-empty and matches the registered device pattern (for example MCH-014)	Reject: invalid device_id
V-03	timestamp format	Parseable ISO 8601 UTC	Reject: invalid timestamp
V-04	timestamp value	Not later than arrival time plus 5 seconds (clock-skew allowance)	Reject: future timestamp
V-05	temperature	Between -20 and 200 °C	Reject: temperature out of range
V-06	pressure	Between 0 and 20 bar	Reject: pressure out of range
V-07	vibration	Between 0 and 50 mm/s	Reject: vibration out of range
V-08	humidity (optional)	If present, between 0 and 100 %	Reject: humidity out of range
V-09	machine_status	One of RUNNING, IDLE, WARNING, FAULT, OFFLINE	Reject: invalid status
V-10	Duplicate	The same event_id was already seen within the last 10 minutes	Drop the duplicate (first copy is kept)
V-11	Lateness	Arrival time minus event time is more than 60 seconds	Store the record, flag it late, exclude it from window aggregates; alerts still apply

The completeness check for silent devices is not a rule on incoming readings. It is the Last Seen check in Section 3.7.4.

A reading outside the plausible physical range (V-05 to V-07) is an invalid record and is rejected. A reading inside the plausible range but above a threshold is a valid abnormal reading and creates an alert. For example, a temperature of 250 °C is rejected, while 105 °C is a Critical alert. Every rejected record keeps the original event content, the rule that failed and the time of rejection.

Deduplication method and its limits. The simulator gives every event a unique event_id. The Stream Analytics query keeps the first event for each event_id and drops later copies that arrive within a 10-minute window (using a limited-duration comparison on event_id). This does not guarantee exactly-once processing in every situation: a copy that arrives after the window, or after a job restart in a way that loses the query state, may not be detected. To make this visible, a Python check runs after each test and counts repeated event_ids in the stored valid records. The controlled test (replay inside the window) targets 0% duplicates.

3.7.2 Transformation rules

The pipeline adds the following fields to each processed record:

Derived field	Meaning
is_valid	True or false, based on the validation rules
rejection_reason	The failed rule, when is_valid is false
arrival_time	Time at which Event Hubs received the event
is_late	True when arrival time minus event time is more than 60 seconds
alert_flag	True when at least one threshold is exceeded
alert_severity	None, Warning or Critical (the highest among violated measurements); calculated, never copied from machine_status
alert_reason	Which measurement exceeded which limit
processing_time	Time at which the pipeline processed the event

- Window aggregates contain device_id, window start, window end, and the average, minimum, maximum and count for temperature, pressure and vibration. Windows use event time.
- The machine health summary contains device_id, reported machine_status, alert_severity, latest readings, number of violations, last_seen_time and the possibly_offline flag.

3.7.3 Anomaly-detection rules

The project uses explicit threshold-based detection only, with the Warning and Critical levels defined in Section 3.3.2. Machine learning is out of scope.

- **AD-01:** a value above the Warning level and not above the Critical level creates a Warning alert.
- **AD-02:** a value above the Critical level creates a Critical alert.
- **AD-03:** a value exactly equal to a limit is not a violation (the comparison is strictly above).
- **AD-04:** each measurement is evaluated independently, and the event severity is the highest one.
- **AD-05:** only valid records are evaluated.
- **AD-06:** threshold values are placeholders for the simulation and must be configurable (FR-19).
- **AD-07:** the alert depends only on the readings. It does not depend on machine_status, so a device reporting RUNNING with a Critical reading still produces a Critical alert.

Boundary examples for temperature (the same pattern applies to pressure and vibration):

Temperature	Result
85.0 °C	No alert (equal to limit)
85.1 °C	Warning
100.0 °C	Warning
100.1 °C	Critical
250 °C	Rejected as invalid (out of range)

3.7.4 Offline detection (Last Seen check)

Azure Stream Analytics does not emit a new event solely because a device becomes silent. A separate lightweight Python health-check process is therefore used for this Should requirement (FR-21).

1. The simulator maintains a small shared device-state file (JSON), updated atomically after each acknowledged send, with device_id and last_seen_time (UTC). This represents producer-observed activity, not independently confirmed Stream Analytics receipt.
2. Every 15 seconds the Python checker reads that file, compares current UTC time to last_seen_time and marks a device possibly_offline when the gap exceeds 30 seconds. Devices are pre-registered so a device that never sends can also be detected.

3. The checker writes its result to a separate health output consumed by the dashboard. For cloud-observed offline status, a later extension can derive last-seen from processed events instead.

This feature remains Should priority; the demo documents any limitations.

3.8 Dashboard and Alerts Requirements

3.8.1 Dashboard pages and content

The live dashboard is a core deliverable. The technology behind it (Section 2.7.1) does not change the content below. Historical pages are built in Power BI on the stored files.

Page or section	Content	Main users	Priority
Plant overview	Machine status tiles (reported status) with a separate alert badge; count of active alerts; count of machines in Fault	Operators, Plant Managers	**Must**
Live readings	Latest temperature, pressure and vibration per machine; average temperature over a time window	Operators, Engineers	**Must**
Alert panel	Recent alerts with machine, measurement, value, limit, severity and time	Operators	**Must**
Device trends	5-minute average, minimum and maximum per device over time	Maintenance Engineers	**Should**
Historical analysis	Alert frequency per machine and common causes of alerts	Analysts	**Nice**
Data quality	Count and reasons of rejected records; possibly-offline devices	Data Engineers	**Nice**
Historical comparison	Current behavior against the machine's historical average	Managers, Analysts	**Nice (optional extension)**

3.8.2 Alert behavior

- **AL-01:** every alert record created by the pipeline shall appear on the alert panel.
- **AL-02:** alerts shall show severity, with Critical visually more prominent than Warning.
- **AL-03:** the alert panel shall show the most recent alerts first.
- **AL-04:** the alert panel shall show the measured value and the limit it exceeded, so the operator understands the reason.
- **AL-05:** invalid records shall not raise operator alerts.
- **AL-06 (Should):** alerts shall not flood the operator. Repeated violations from the same machine and measurement should be grouped or summarized.

Alerts in this prototype are displayed on the dashboard. Sending email, SMS or other external notifications is not required.

3.9 Prioritization and Traceability

3.9.1 Summary by priority

Priority	Requirements
Must (29)	FR-01, 02, 04, 07, 08, 10, 11, 12, 13, 14, 16, 17, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 38, 39, 40, 41, 42
Should (9)	FR-05, 06, 09, 15, 19, 21, 32, 33, 37
Nice (4)	FR-03, 34, 35, 36

3.9.2 Traceability to use cases

Every use case is covered by at least one functional requirement.

Use case	Related functional requirements
UC-01 Abnormal reading detection	FR-16, 17, 18, 30, 38, 42
UC-02 Device trend review	FR-20, 27, 33
UC-03 Real-time plant overview	FR-22, 28, 29, 31, 41, 42
UC-04 Compare against history (optional)	FR-27, 36
UC-05 Quality retrieval by time period	FR-26, 27, 40
UC-06 Alert frequency analysis	FR-24, 27, 34
UC-07 Review rejected records	FR-12, 25, 35, 40
UC-08 Executive KPIs	FR-31, 38

3.10 Non-Functional Requirements and Quality Standards

3.10.1 Purpose and reference framework

Functional requirements describe what the pipeline does; non-functional requirements (NFRs) describe how well it must do it. In a streaming system the value of an event decays with time: an alert that arrives ten minutes late only records a machine that has already been damaged. Every requirement below has a metric, an acceptance threshold, a justification and a test method, so that each claim can be verified instead of argued. All tests are controlled test runs at prototype scale (Section 1.6); they are not long production measurements.

Requirements are organized with the ISO/IEC 25010 quality model. Reliability targets are expressed as service-level indicators (SLIs, the measured value) and objectives (SLOs, the target for it).

ISO 25010 characteristic	Covered in
Performance efficiency	Performance (3.10.3) and Scalability (3.10.4)
Reliability	Reliability (3.10.5)
Security	Security (3.10.6)
Functional suitability (correctness)	Data Quality (3.10.7)
Usability (ease of use)	Usability (3.10.8)
Maintainability	Operability (3.10.9)

3.10.2 Workload assumptions

Parameter	Value	Reasoning
Development runs	5–10 machines, 1 event/s each	Cheapest way to build and debug
Baseline (demonstration and acceptance tests)	50 machines, 1 event/s each, 50 events/s, in sessions of about 30 minutes, if the budget allows	Simulator baseline used in Section 3.3.3
Optional higher-load test	100–200 events/s for 10 minutes	Shows headroom without a large cost
Event size	About 300 bytes (JSON)	event_id, device ID, timestamp, three readings, humidity and status
Ingress at baseline	About 15 KB/s	50 events x 300 bytes

Capacity beyond these figures is not tested in this project. It is described only under Future Extensions (Section 3.11.2).

3.10.3 Performance

ID	Requirement	Metric	Acceptance threshold	Justification	Test method
NFR-P1	Reading latency: from event timestamp to the processed valid record being written to the output store	P95 latency	5 s or less	An operator must see a critical alert while the machine can still be stopped	Compare event timestamp with output write time over a 30-minute run at baseline
NFR-P2	Alert latency for threshold violations	P99 latency	10 s or less	Alerts are the highest-value output	Inject 100 known anomalies and measure time to alert record
NFR-P3	Sustained ingestion at baseline	events/s	50 events/s for 30 min with no growing backlog, if the budget allows; optional 100–200 events/s for 10 min	Covers the prototype workload	Load test with the simulator; watch backlogged input events
NFR-P4	Processing headroom	SU utilization	80% or less during the baseline test	Above about 80% latency rises and bursts cannot be absorbed	Azure Monitor: SU % utilization
NFR-P5	Live dashboard refresh for readings and alerts (Must)	Time from output update to visible dashboard update	10 s or less with the live method chosen by the proof of concept. If a method cannot reach it, the measured value is documented and agreed with the supervisor	Operators act on what the dashboard shows	Observe and record update times
NFR-P6	Window result delay: from window end until the 1-minute or 5-minute aggregate is written	Delay after window end	90 s or less (60 s late tolerance plus 30 s processing)	A window cannot close before its tolerance period ends	Compare window end time with output write time

> Latency targets are proposed acceptance goals, not guaranteed Azure service timings. A single reading or alert is measured separately from a completed window. Run an early 5–10-device proof of concept, record observed P95/P99 and output-write timestamps, then test the 50 events/s baseline. If the targets cannot be met with the approved budget, document results and seek supervisor approval before changing targets; do not add costly services just to meet them.

Why percentiles: an average hides slow outliers. A system with a one-second average can still leave 5% of events waiting 20 seconds.

3.10.4 Scalability

Large-scale testing is outside the prototype. The design keeps the option open, and the tests below are small.

ID	Requirement	Metric	Acceptance threshold	Justification	Test method
NFR-S1	Parallelism by device	Partition key	Events of one device always in one partition	Preserves per-device order and spreads load	Verify all events of a device land in the same partition
NFR-S2	Tolerance to uneven device load	Partition share	No partition above 2x the average load	One chatty machine must not create a hot partition	Run one device at 10x rate; inspect per-partition metrics

The number of Event Hubs partitions limits consumer parallelism and is fixed at creation on the Standard tier, so it is chosen with future growth in mind. Growth by configuration is a Future Extension (Section 3.11.2).

3.10.5 Reliability

ID	Requirement	Metric	Acceptance threshold	Justification	Test method
NFR-R1	No data loss when the processing job restarts	Data loss rate	0% of events acknowledged by Event Hubs, within the retention period	Lost readings can hide a developing fault	Send a known count, stop and restart the job midway, compare counts at the output
NFR-R2	Duplicate handling under at-least-once delivery	Duplicate rate in stored valid data	0% in the controlled test (replay inside the 10-minute window), with a post-run check; no exactly-once guarantee is claimed	Replays can occur after a failure	Replay a window of events with the same event_ids and count repeated event_ids at the output
NFR-R3	Pipeline availability during test and demo sessions	Successful operating time divided by scheduled session time	99.5% or more	Monthly uptime would need continuous operation and monitoring, which the budget does not allow	Session log and Azure Monitor metrics
NFR-R4	Late and out-of-order data	Accepted lateness	Events up to 60 s late land in the window of their event time; later events are stored, flagged and excluded from aggregates	Network delays and device buffering are normal in IoT	Send events with delayed timestamps and check window results
NFR-R5	Device disconnection detection (Should)	Silence period	A device with no readings for more than 30 s is flagged possibly offline by the Last Seen check	A silent sensor is itself a failure	Stop one simulated device and check the health view
NFR-R6	Recovery objectives	RTO / RPO	RTO 15 min or less; RPO close to 0 for events still inside the retention period	Event Hubs retention lets the job resume from its last checkpoint	Timed job-restart test from NFR-R1

3.10.6 Security

The baseline below is required. The optional items add setup effort and possible cost and are listed separately.

ID	Requirement	Metric	Acceptance threshold	Justification	Test method
NFR-SEC1	Encryption in transit	Connections using TLS	TLS 1.2 or higher on 100% of connections	Prevents interception or tampering	Configuration review; a non-TLS connection attempt must fail
NFR-SEC2	Encryption at rest	Storage encryption	Enabled on Blob Storage (ADLS Gen2) with platform-managed keys	Protects historical data	Check storage account settings
NFR-SEC3	Device authentication and least privilege for producers	Credential scope	Separate credential per producer; SAS policy with Send permission only	A compromised device must not read data or affect others	Attempt a Listen operation with a device credential; it must be denied
NFR-SEC4	No secrets in code	Secrets in repository	0 hard-coded secrets; credentials kept in environment variables or a local file excluded from GitHub	Leaked keys are a leading cause of cloud breaches	Repository secret scan; review of settings
NFR-SEC5	Least-privilege access	Role assignments	Each component and team member holds only the roles needed	Limits the blast radius of a compromise	Access review of role assignments
NFR-SEC6	Evidence logs	Log retention	Keep only the access and configuration logs needed as test evidence	Avoids extra storage and monitoring cost	Check diagnostic settings

Optional hardening (not required for acceptance): Managed Identity and Azure Key Vault for service-to-service secrets, and access and configuration logs kept for 30 days or more. These are listed under Future Extensions.

The OWASP IoT Top 10 is used as a checklist for device-side risks such as weak credentials and insecure network services. Device-side controls are limited to the simulator's credentials; physical device security is out of scope.

3.10.7 Data quality

These targets measure different things and are not in conflict. NFR-DQ1 measures whether good records are accepted. NFR-DQ2 to DQ6 measure whether bad records are caught. NFR-DQ7 splits detection into fixed threshold cases (expected to be exact) and statistical recall and false-positive rate on injected anomalies.

ID	Dimension	Requirement	Acceptance threshold	Test method
NFR-DQ1	Completeness / acceptance of valid records	Known valid test records reach the valid store	99% or more of known valid records accepted	Validation counters: input = valid + rejected + duplicates + malformed
NFR-DQ2	Validity	Values inside physical ranges; status in the allowed list; invalid records rejected	100% of predefined invalid test cases rejected; 0 invalid records in the valid store	Inject invalid records; all must be rejected
NFR-DQ3	Timeliness	Timestamp not in the future; late events flagged	0 future-timestamp records in the valid store; 100% of late records flagged and excluded from aggregates	Inject future and late timestamps
NFR-DQ4	Uniqueness	No repeated event_id in the valid store	0% duplicates in the controlled test	Query for repeated event_ids
NFR-DQ5	Consistency	Records follow the agreed schema and types	100% of valid records conform	Schema validation at input
NFR-DQ6	Traceability	Well-formed rejected records kept with a reason; malformed messages kept in the raw store	100% of rejected records stored with a reason; 100% of malformed test messages found in the raw store	Compare rejected count with the rejected store; compare raw count
NFR-DQ7	Detection accuracy	Part A: predefined threshold and boundary test cases. Part B: injected anomalies	Part A: 100% of cases correct. Part B: recall 95% or more and false positives 5% or less	Part A: compare with expected outcomes. Part B: evaluation script against labelled anomalies

3.10.8 Usability

The dashboard is used by operators who have little time and no training in the tool. These requirements keep it easy to read and easy to run. The targets are proposed for a small prototype test, not for a formal usability study.

ID	Requirement	Metric	Acceptance threshold	Justification	Test method
NFR-U1	A first-time user finds the machine with a Critical alert on the overview page	Time to name the machine	At least 4 of 5 testers within 30 s, with no help	Operators must react quickly and cannot be trained for every screen	Observed test: five testers, one Critical machine among ten, scripted scenario
NFR-U2	Severity is clear without relying on color alone	Alert rows with a text label	100% of alert rows show a text label (Warning or Critical) as well as a color	Color alone fails for color-blind users and on poor screens	Design review and a grayscale screenshot check
NFR-U3	The overview fits one screen	Must items visible without scrolling	Status tiles, active-alert count, machines in Fault and the alert panel are visible together at 1920 x 1080	Managers want the plant state at a glance	Check the dashboard on a 1080p screen
NFR-U4	A new person can run the project from the README	Time to first live data	Someone who did not write the code starts the simulator, sees data on the live dashboard and stops all resources in 60 min or less	Reproducibility and safe cost control	One observed run using only the README

3.10.9 Operability

ID	Requirement	Acceptance threshold
NFR-O1	Monitoring	Basic Azure Monitor charts for input backlog, SU utilization, output errors and input deserialization errors during test sessions
NFR-O2	Reproducible deployment	Infrastructure described as scripted steps (CLI or template), so another person can rebuild the pipeline and delete it after use

3.10.10 Quality acceptance criteria

The most important requirements are written as Given / When / Then scenarios that can be run during the demonstration.

ID	Scenario	Given	When	Then
AC-1	Restart and deduplication (NFR-R1, R2)	10,000 uniquely identified events are sent, including a replayed block within the 10-minute deduplication window	The Stream Analytics job is stopped and restarted midway, then all outputs finish writing	Reconcile acknowledged sends, raw messages, valid and rejected outputs and processing errors. Verify 0 duplicate event_ids for the replayed block within the 10-minute window during uninterrupted processing. After restart, measure and report any duplicate or missing IDs; do not claim exactly-once or guaranteed 0% duplicates across restarts
AC-2	Sustained baseline (NFR-P1, P3)	The simulator sends 50 events/s for 30 minutes, if the budget allows	Latency is measured per event	P95 reading latency is 5 s or less and the input backlog does not grow continuously
AC-3	Abnormal-event detection (NFR-P2, DQ7)	The simulator injects 100 labelled anomalies and a fixed set of boundary cases	The pipeline processes the stream	All boundary cases are correct; at least 95 anomalies are flagged, each alert within 10 s, and false alerts stay at or below 5%
AC-4	Invalid-data handling (NFR-DQ1 to DQ6)	The input contains null fields, negative pressure, unknown status values and future timestamps, plus a few malformed JSON messages	The validation rules run	None of the invalid records reaches the valid store; each well-formed one is stored in the rejected store with a reason; each malformed one is found in the raw store
AC-5	Unauthorized access is denied (NFR-SEC1, SEC3)	A device credential holds Send-only permission	It tries to read from the event hub or to connect without TLS	Both attempts are refused
AC-6	Window result delay (NFR-P6, R4)	The simulator sends events for 3 devices for 10 minutes, including some events 30 s late and some 90 s late	1-minute and 5-minute windows close	Aggregates are written 90 s or less after window end; the 30-s-late events are counted; the 90-s-late events are stored, flagged and not counted
AC-7	Status and alert are independent (FR-28, AD-07)	A simulated device reports RUNNING and sends a temperature above the Critical level	The event is processed	The record keeps machine_status = RUNNING, alert_severity = Critical, and the alert appears on the panel
AC-8	Live dashboard (FR-42, NFR-P5)	The live method from the proof of concept is running and the simulator is sending events	An abnormal reading is injected	The alert is visible on the live dashboard within 10 s of the record being written, or the measured value is documented
AC-9	Ease of use (NFR-U1 to U4)	Five testers who did not build the dashboard, and one person who follows only the README	They find the Critical machine, and the README user runs the project	At least 4 of 5 find it within 30 s; text labels are on all alerts; the overview fits one screen; the README run takes 60 min or less

Scaling scenarios are listed under Future Extensions (Section 3.11.2).

3.10.11 Trade-offs

- **Latency vs window correctness.** A longer lateness tolerance makes aggregates more accurate but delays final window results. One minute is the compromise chosen here, and it is the reason window results are measured separately from single readings.
- **Delivery guarantees vs complexity.** Event Hubs provides at-least-once delivery. Deduplicating on event_id within a limited window is simple and testable, but it is not an exactly-once guarantee.
- **Availability vs cost.** Zone redundancy, geo-disaster recovery and continuous operation raise availability and cost. The prototype is measured over test sessions only.
- **Headroom vs cost.** Keeping SU utilization at or below 80% means paying for spare capacity, but it prevents latency spikes during bursts.
- **Security vs convenience.** Per-device Send-only credentials need more setup than one shared key, but a leaked shared key would expose the whole pipeline. Managed Identity and Key Vault are the next step.

- **Strict validation vs data loss.** Rejecting borderline records protects quality but can discard useful readings. Keeping them in the rejected store (and everything in the raw store) allows later review.
- **Live dashboard speed vs cost.** A faster live method can need an extra paid service. The cheapest method that reaches the refresh target is chosen.

3.10.12 Verification notes

Platform limits, tiers, prices and service-level figures change over time. Two points were checked against Microsoft documentation while preparing this edition: Event Hubs Capture is available from the Standard tier and not on Basic [8], and the Stream Analytics Power BI output is deprecated for new customers [6][7]. Before implementation, the following should still be checked: Event Hubs throughput limits, retention and partitions per tier; published SLA percentages for Event Hubs, Stream Analytics and Blob Storage; delivery guarantees of Stream Analytics for each output type; the output path pattern and custom partition options of the Blob Storage output; how Stream Analytics treats deserialization errors and late events; and the licence and limits of the live-dashboard method chosen in the proof of concept. The targets in this document are design goals for the project workload, not vendor guarantees.

3.11 Assumptions and Future Extensions

3.11.1 Assumptions

- All data is simulated, and all devices share one event schema.
- The number of devices, event rates, thresholds and plausible ranges are realistic placeholders to be reviewed before final use.
- Alerts are shown on the dashboard only.
- The Azure budget, the planned running hours and the live-dashboard method are confirmed before the first deployment (Section 1.6 and Section 2.7.1).
- Tests are controlled runs at prototype scale; no long-term production measurements are made.

3.11.2 Future Extensions

The figures and features in this section are outside the prototype. They are not requirements of this project and are not tested.

Area	Future extension
Future scalability: capacity	A 500-machine design capacity at 500 events/s on average, a peak of 1,000 events/s, and growth of 10x (10,000 events/s). Continuous 24-hour operation would produce about 1.3 GB of raw data per day at the baseline event size.
Future scalability: load tests	Load tests at 1,000 events/s sustained for 30 minutes, scaling to 10,000 events/s by increasing partitions, throughput units and Streaming Units without code changes, with a design review confirming that growth needs configuration changes only.
Availability	Monthly 99.5% uptime with continuous monitoring, zone redundancy and disaster recovery. A 99.5% target allows about 3.6 hours of downtime in a 30-day month. The composite availability of a pipeline is roughly the product of its components' SLAs, so every added service lowers the ceiling.
Security hardening	Managed Identity and Azure Key Vault, and access and configuration logs kept for 30 days or more.
Detection and analysis	Historical-baseline comparison with live data (UC-04), statistical anomaly detection and machine-learning-based prediction.
Architecture and alerts	Edge processing, a serving database such as Azure SQL or Azure Data Explorer for interactive history queries, and external notifications (email, SMS) for alerts.
Live dashboard	Migration to Fabric Real-Time Intelligence if the first live method is retired, and a production dashboard with user sign-in.

Part 3 (continued)

3.12 Supplementary Non-Functional Requirements

The brief asks for scalability requirements when the number of sensors grows and for accuracy as a data-quality standard. The requirements below extend Section 3.10.4 and Section 3.10.7 without changing their existing IDs. They stay at prototype scale (Section 3.10.2); larger loads remain under Future Extensions (Section 3.11.2).

ID	Requirement	Metric	Acceptance threshold	Justification	Test method
NFR-S3 (Should)	The pipeline keeps its latency targets when the number of sensors increases	P95 reading latency and SU utilization at each device count	At 10, 25 and 50 devices (1 event/s each, same configuration): P95 is 5 s or less (NFR-P1) and SU utilization is 80% or less (NFR-P4). No code change between steps.	Shows that more sensors need more capacity, not a redesign	Short runs (about 10 minutes each, budget permitting) at the three device counts; compare P95 and SU % in Azure Monitor
NFR-S4 (Should)	A new sensor can join without changing the pipeline	Changes needed to add a device	A new device_id that matches the registered pattern (V-02) is accepted and processed with no change to the query or the dashboard.	Machines are added over time	Add one device during a run; check that its readings reach the valid store, the aggregates and the dashboard
NFR-DQ8	Accuracy of stored values	Stored values equal the values sent	100% of valid records keep timestamp, temperature, pressure, vibration and unit exactly as sent (same value and precision, no unit conversion).	A wrong value gives a wrong alert or a wrong trend	The simulator writes a ground-truth file of what it sent; a script compares it with the valid store by event_id

Accuracy has two parts in this document: value accuracy (NFR-DQ8) and detection accuracy (NFR-DQ7).

Acceptance scenarios

ID	Scenario	Given	When	Then
AC-10	Sensor growth (NFR-S3, S4)	The simulator runs at 10, 25 and 50 devices in turn, and one extra registered device is added during the last run	The load increases at each step	P95 reading latency stays at 5 s or less and SU utilization at 80% or less; the added device appears with no pipeline change
AC-11	Value accuracy (NFR-DQ8)	The simulator has sent a known set of valid events and kept a ground-truth file	The valid store is compared with the file by event_id	100% of valid records match the values sent; any difference is listed with its event_id

Part 4: Evaluation and Improvement

This part holds the lecturer's feedback, the suggested improvements and the final grading criteria.

> **Status**
> The lecturer has not evaluated the project yet. The cells in Sections 4.1 and 4.3 are left empty on purpose and are filled only when the lecturer gives the actual comments and the marking scheme. No scores or comments have been estimated or assumed.

4.1 Lecturer Feedback and Evaluation

Item	Entry
Lecturer name	
Date of feedback	
Document version reviewed	

Section reviewed	Lecturer's comment	Score / rating	Team response
Part 1: Project Planning & Management			
Part 2: Literature Review			
Part 3: Requirements Gathering			
Overall evaluation			

4.2 Suggested Improvements

4.2.1 Review findings and how they were addressed

The table lists the points raised in the internal review of the previous version and where each one is handled. Rows 14 to 17 record the decisions applied in this edition.

#	Finding	Resolution in this version	Section
1	Literature Review lacked feedback, improvement and grading sections	Added Part 4 with these sections; lecturer entries left for the lecturer's real input	4.1 to 4.3
2	Deduplication target conflicted (0.1% versus none) and was not guaranteed across restarts	Unique event_id from the simulator; 10-minute detection window; target 0% in the controlled test; no exactly-once claim; post-run check	3.3.1, 3.7.1, NFR-R2, AC-1
3	A 5-second latency target does not fit window aggregation	Latency split into reading and alert latency (5 s and 10 s) and window result delay (90 s after window end)	1.5, 3.10.3, AC-6
4	Raw data path was not clear	Independent raw output: Event Hubs Capture as the primary path, simulator audit log as a documented fallback with stated limits; malformed JSON handled	3.3.4, FR-23, FR-40, AC-4

#	Finding	Resolution in this version	Section
5	Machine status was ambiguous	machine_status (device), alert_severity (analysis) and possibly_offline (silence) separated	3.3.1, AD-07, AC-7
6	Power BI live integration was treated as guaranteed	Live dashboard made a Must requirement; proof of concept by M4 compares methods because the Stream Analytics Power BI output is deprecated for new customers	2.7.1, FR-41, FR-42, AC-8
7	Offline detection cannot be a query condition	Should priority and a separate Last Seen check every 15 s	3.7.4, FR-21
8	Blob Storage is not directly queryable	Retrieval through folder layout by device and date, read with Python or Power BI	3.3.4, FR-26
9	Timestamp validation conflicted with late-event handling	Event time, arrival time and processing time defined; late events stored, flagged and excluded from aggregates	3.3.5, V-04, V-11
10	Data-quality KPIs were not clearly separated	Acceptance of valid records, rejection of invalid ones, threshold cases and recall/false positives defined separately	1.5, 3.10.7
11	Some requirements were too heavy or costly for a student prototype	5–10 devices for development; 50 events/s baseline if the budget allows; 100–200 events/s optional; every higher figure moved to Future Extensions; availability measured over test sessions; Managed Identity and Key Vault optional	3.10, 3.11.2
12	No budget estimate	Budget cap, running hours, cost alerts and shutdown plan added	1.6
13	KPIs and non-functional requirements were complex; User Stories were not separate	KPI table simplified; user stories added	1.5, 3.4.2
14	Timeline dates were inconsistent between the Gantt chart and the milestone table	All dates follow the confirmed schedule: 2 Oct to 4 Dec 2026, implementation 7–30 Nov	1.2
15	Usability was not covered by the quality requirements	Usability requirements NFR-U1 to U4, user story US-09 and acceptance scenario AC-9 added	3.10.8, 3.4.2, AC-9
16	Literature review sources were not listed with links	Numbered references with links added, including the EMSTEEL link	References
17	Capture tier and Power BI connector status were assumed	Checked against Microsoft documentation and recorded	3.10.12, 2.7.1

4.2.2 Further improvements the team may consider

See Future Extensions (Section 3.11.2). Any improvement requested by the lecturer is added to the table in Section 4.1 and to this section after the lecturer's feedback is received.

4.3 Final Grading Criteria

The criteria and weights are not known to the team yet. They are copied exactly from the lecturer's marking scheme when it is provided.

Criterion	Weight / maximum mark	Where it is covered in this document	Team self-check
Total			

References

Numbers match the citations in the text. Links were opened when this edition was prepared (20 October 2026), except where marked.

#	Source	Link	Used in
[1]	Microsoft Customer Story — Grupo Bimbo (GBConnected, factory-line monitoring with Azure IoT)	https://customers.microsoft.com/story/1496159330015555860-grupo-bimbo-consumer-goods-azure-iot	2.1.1
[2]	Microsoft Customer Story — XTO Energy taps into IoT and the cloud to optimize operations and drive growth with Azure and Dynamics 365	https://microsoft.com/en/customers/story/709893-exxon mobil-mining-oil-gas-azure	2.1.2
[3]	Microsoft Customer Story — 3M manufacturing plant uses data analytics to increase efficiency and cost savings with Azure SQL Edge	https://www.microsoft.com/en/customers/story/844496-3m-manufacturing-azure-sql-edge	2.1.3
[4]	Microsoft Customer Story — EMSTEEL (Microsoft Fabric Real-Time Intelligence)	https://www.microsoft.com/en/customers/story/26137-emsteel-microsoft-fabric	2.1.4
[5]	NRI North America, Machine Monitoring IoT Solution with Azure Services and Power BI, 11 June 2019	https://nri-na.com/machine-monitoring-iot-solution-azure-services-power-bi/	2.5.2
[6]	Microsoft Learn — Power BI output from Azure Stream Analytics (deprecation notice and limits)	https://learn.microsoft.com/en-ca/Azure/stream-analytics/power-bi-output	2.7.1, 3.10.12
[7]	Retirement notice AZ466774: Azure Stream Analytics Power BI output connector (copy of the Microsoft message-center post)	https://mc.merill.net/message/AZ466774	2.7.1
[8]	Microsoft Learn — Compare Azure Event Hubs tiers (Capture support and retention per tier)	https://learn.microsoft.com/en-us/azure/event-hubs/compare-tiers	1.6, 3.3.4

Customer stories [1]–[5] are vendor or consultancy accounts of intended benefits, not independent evaluations (Section 2.4).